

1. 词法分析

冯洋

南京大学计算机学院

fengyang@nju.edu.cn

提纲

- 词法分析的定义与作用
- 词法单元的描述（正则表达式）
- 词法单元的识别（状态转换图/有穷自动机）
- 词法分析器生成工具及设计

词法分析的定义

- 词法分析是读入源程序的输入字符、将它们组成**词素**，生成并输出一个**词法单元序列**，每个词法单元对应于一个词素
- 常见的做法
 - 由语法分析器调用，需要的时候不断读取、生成词法单元
 - 可以避免额外的输入输出
- 在识别出词法单元之外，还会完成一些不需要生成词法单元的简单处理，比如删除注释、将多个连续的空白字符压缩成一个字符等

词法分析的定义

- 一个简单的例子

`position = initial + rate * 60`

词法分析的定义

- 一个简单的例子

position = initial + rate * 60

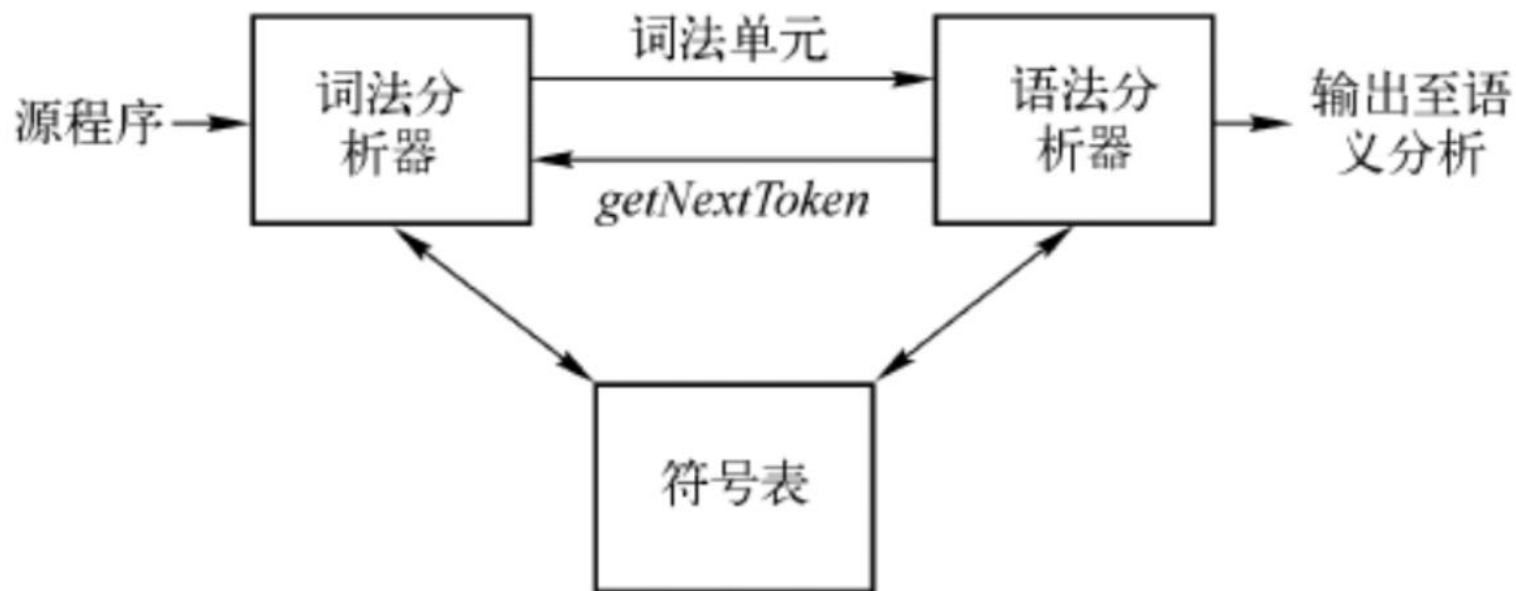
词法单元序列: <id,1> <=,> <id, 2> <+,> <id,3> <*,> <number, 4>

词法分析的定义

- 一个简单的例子

position = initial + rate * 60

词法单元序列: <id,1> <=, > <id, 2> <+, > <id,3> <*, > <number, 4>

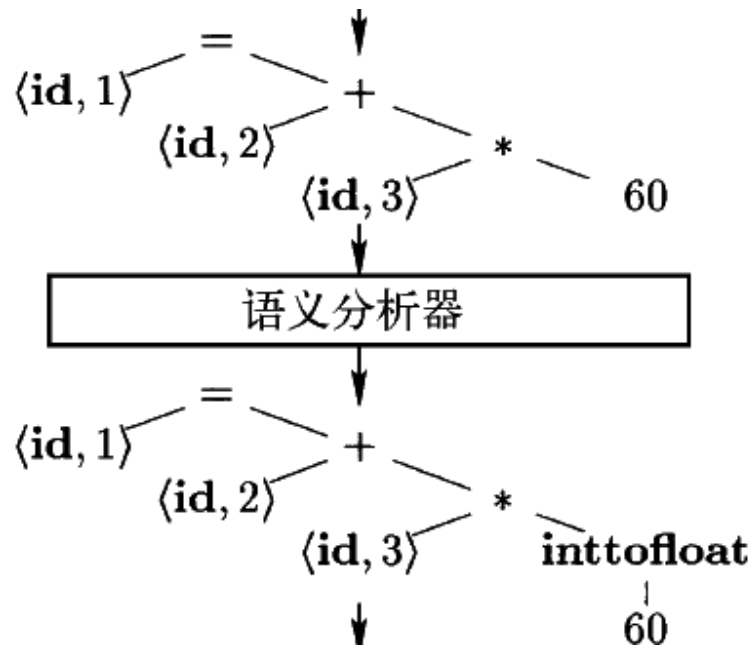
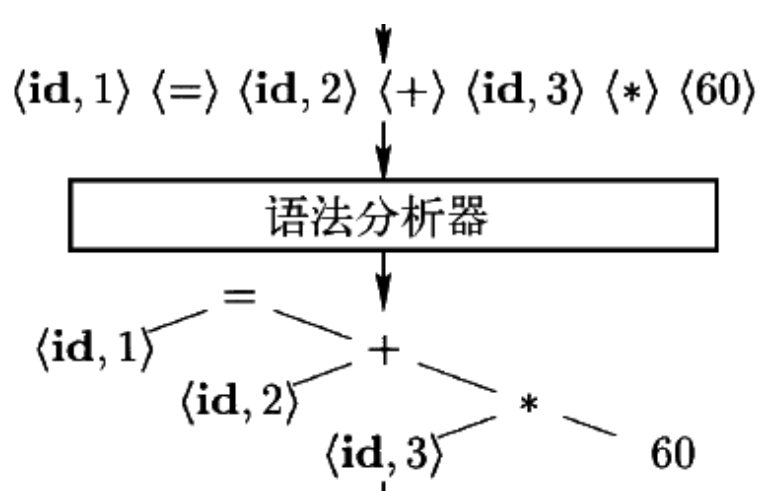


词法分析的定义

- 一个简单的例子

position = initial + rate * 60

词法单元序列: <id,1> <=,> <id, 2> <+,> <id,3> <*,> <number, 4>



词法分析相关概念

- 词素 (Lexeme)

- 源程序中的字符序列，它和某类词法单元的模式匹配，被词法分析器识别为该词法单元的实例。

- 词法单元 (Token)

- 包含单元名 (Token-name) 和可选的属性值(attribute-value)
- 单元名是表示某种词法单位抽象符号。语法分析器通过单元名即可确定词法单元序列的结构。

词法分析相关概念

词法单元	非正式描述	词素示例
if	字符 i, f	if
else	字符 e, l, s, e	else
comparison	< 或 > 或 <= 或 >= 或 == 或 !=	<=, !=
id	字母开头的字母 / 数字串	Pi, score, D2
number	任何数字常量	3.14159, 0, 6.02e23
literal	在两个 "之间, 除" 以外的任何字符	"core dumped"



词法分析相关概念

- 词素通常用以表示一类符号：整数，操作符，标识符...
- 因此需要用一种方式描述某一类的词素

词法分析相关概念

- 词素通常用以表示一类符号：整数，操作符，标识符...
- 因此需要用一种方式描述某一类的词素
- 模式匹配：
 - 一个模式可能匹配多个词素
 - 必须通过属性来传递附加的信息
 - 属性值将被用于语义分析、代码生成等阶段
 - 不同的目的需要不同的属性
 - 属性值通常是一个结构化数据（词素、类型、第一次出现的位置）

词法分析相关概念

词素



词法单元

第一步：如何描述词素？

Specification

词法分析相关概念

词素



词法单元

第一步：如何描述词素？

Specification

模式 (Pattern)

- 词法单元对应的词素可能具有的形式
- 可以用正则表达式来表示

正则表达式基础

- 字母表：一个有限的符号集合
 - ASCII
 - Unicode
 - 典型的字母表包括字母、数位和标点符号
- 问题：二进制的符号表是什么？
- 问题：汉语的符号表是什么？

正则表达式基础

- 字母表：一个有限的符号集合
 - ASCII
 - Unicode
 - 典型的字母表包括字母、数位和标点符号
- 问题：二进制的符号表是什么？ $\{0,1\}$
- 问题：汉语的符号表是什么？

正则表达式基础

- 串：字母表中符号组成的一个有穷序列
 - 串 s 的长度 $|s|$
 - 空串 ε ，长度为0的串
- 语言：给定字母表上一个任意的可数的串的集合
 - 语法正确的C程序的集合
 - 英语、汉语、法语.....
 - 机器码

正则表达式基础

- 和串有关的术语 (banana)

- 前缀：从串的尾部删除0个或多个符号后得到的串 (ban、banana、 ϵ)
- 后缀：从串的开始处删除0个或多个符号后得到的串 (nana、banana、 ϵ)
- 子串：删除串的某个前缀和某个后缀得到的串 (banana、nan、 ϵ)
- 真前缀、真后缀、真子串：既不等于原串，也不等于空串的前缀、后缀、子串
- 子序列：从原串中删除0个或者多个符号后得到的串 (baan)

正则表达式基础

- 串的运算

- 连接(concatenation): x 和 y 的连接时把 y 附加到 x 的后面形成的串, 记作 xy 。

- $x=\text{dog}$, $y=\text{house}$, $xy=\text{doghouse}$

- 指数运算(幂运算): $s^0=\varepsilon$, $s^1=s$, $s^i=s^{i-1}s$;

- $x=\text{dog}$, $x^0=\varepsilon$, $x^1=\text{dog}$, $x^3=\text{dogdogdog}$

正则表达式基础

- 语言上的运算

- L 和 M 的并:

L 和 M 的并

$$L \cup M = \{ s \mid s \text{ 属于 } L \text{ 或者 } s \text{ 属于 } M \}$$

L 和 M 的连接

$$LM = \{ st \mid s \text{ 属于 } L \text{ 且 } t \text{ 属于 } M \}$$

L 的 Kleene 闭包

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

L 的正闭包

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

- 语言： 串的集合

正则表达式基础

- 语言的一些实例：

- $L = \{A, B, \dots, Z, a, b, \dots, z\}$
- $D = \{0, 1, \dots, 9\}$
- $L \cup D : \{A, B, \dots, Z, a, b, \dots, z, 0, 1, \dots, 9\}$
- LD : 520个长度为2的串的集合
- L^4 : 所有由四个字母构成的串的集合
- L^* : 所有字母构成的集合，包括 ε 。

正则表达式基础

- 语言的一些实例：
 - $L = \{A, B, \dots, Z, a, b, \dots, z\}$
 - $D = \{0, 1, \dots, 9\}$
 - $L \cup D : \{A, B, \dots, Z, a, b, \dots, z, 0, 1, \dots, 9\}$
 - LD : 520个长度为2的串的集合
 - L^4 : 所有由四个字母构成的串的集合
 - L^* : 所有字母构成的集合, 包括 ε 。
- 问题: $L(L \cup D)^*$
- 问题: D^+

正则表达式的定义

- 正则表达式是一种描述词素模式的重要表示方法
- 正则表达式可以高效、简洁地描述处理词法单元时用到的模式类型
- 可以描述所有通过对某个字母表上的符号应用运算符而得到的语言；其定义的集合叫做正则集合(regular set)
- 每个正则表达式 r 可以描述一个语言 $L(r)$ ，也即其定义的正则集合

正则表达式的定义

- 正则表达式可以由较小的正则表达式递归构建（字母表 Σ 上的正则表达式的定义）
 - 基本部分
 - ϵ 是一个正则表达式, $L(\epsilon)=\{\epsilon\}$
 - 如果 a 是 Σ 上的一个符号, 那么 a 是正则表达式, $L(a)=\{a\}$
 - 归纳步骤:
 - 选择: $(r) \mid (s)$, $L((r) \mid (s))=L(r) \cup L(s)$;
 - 连接: $(r)(s)$, $L((r)(s))=L(r)L(s)$;
 - 闭包: $(r)^*$, $L((r)^*)=(L(r))^*$;
 - 括号: (r) , $L((r))=L(r)$
- 运算的优先级: $*$ $>$ 连接符 $>$ $|$
(a)|(b)*(c)可以改写为 $a|b^*c$

正则表达式的定义

- 正则表达式实例

- $\Sigma = \{a, b\}$

- $L(a|b) = \{a, b\}$

- $L((a|b)(a|b)) = \{aa, ab, ba, bb\}$

- $L(a^*) = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

- $L((a|b)^*) = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$

- $L(a|a^*b) = \{a, b, ab, aab, aaab, \dots\}$

正则表达式的定义

- 正则表达式的**等价**:

➤如果两个正则表达式 r 和 s 表示同样的语言, 则 $r = s$

定律	描述
$r s = s r$	是可以交换的
$r (s t) = (r s) t$	是可结合的
$r(st) = (rs)t$	连接是可结合的
$r(s t) = rs rt; (s t)r = sr tr$	连接对 是可分配的
$\epsilon r = r\epsilon = r$	ϵ 是连接的单位元
$r^* = (r \epsilon)^*$	闭包中一定包含 ϵ
$r^{**} = r^*$	* 具有幂等性

正则表达式的定义

- 正则定义：

- 对正则表达式命名，使表示简洁

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

- 各个 d_i 不在字母表 Σ 中，且名字都不同
- 可以认为 d_i 是正则表达式的“名字”
- 每个 r_i 都是 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的正则表达式

正则表达式的定义

- 正则定义：

- 各个 d_i 在 Σ 上的正则表达式如下：

- d_1 的正则表达式即 r_1

- 将 r_2 中的 d_1 替换为 r_1 ，得到 d_2 的正则表达式

-

- 将 r_i 中的 $d_1, d_2, \dots, d_{i-1}$ 替换为各自的正则表达式，得到 d_i 的正则表达式

- 注意：替换的时候不能破坏替换进去的 d_i 的完整性

正则表达式的定义

[lex.token] Tokens

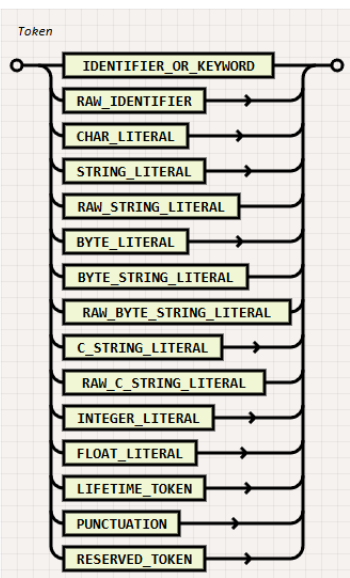
» [lex.token.syntax]

Lexer

Token →

- | IDENTIFIER_OR_KEYWORD
- | RAW_IDENTIFIER
- | CHAR_LITERAL
- | STRING_LITERAL
- | RAW_STRING_LITERAL
- | BYTE_LITERAL
- | BYTE_STRING_LITERAL
- | RAW_BYTE_STRING_LITERAL
- | C_STRING_LITERAL
- | RAW_C_STRING_LITERAL
- | INTEGER_LITERAL
- | FLOAT_LITERAL
- | LIFETIME_TOKEN
- | PUNCTUATION
- | RESERVED_TOKEN

Hide syntax diagram



[ident] Identifiers

nt.syntax]

Lexer

IDENTIFIER_OR_KEYWORD →

XID_Start XID_Continue*

|  XID_Continue+

Rust支持Unicode

XID_Start → <XID_Start defined by Unicode>

XID_Continue → <XID_Continue defined by Unicode>

RAW_IDENTIFIER →  IDENTIFIER_OR_KEYWORD_{except crate, self, super, Self}

NON_KEYWORD_IDENTIFIER → IDENTIFIER_OR_KEYWORD_{except a strict or reserved keyword}

IDENTIFIER → NON_KEYWORD_IDENTIFIER | RAW_IDENTIFIER

RESERVED_RAW_IDENTIFIER → 

Hide syntax diagram

正则表达式的定义

- Pascal无符号数集合，例如：1946, 11.28, 63.6E8, 1.99E-6

$\text{digit} \rightarrow 0 \mid 1 \mid \dots \mid 9$

$\text{digits} \rightarrow \text{digit} \text{ digit}^*$

$\text{optional_fraction} \rightarrow . \text{digits} \mid \varepsilon$

$\text{optional_exponent} \rightarrow (E (+ \mid - \mid \varepsilon)$
 $\text{digits}) \mid \varepsilon$

$\text{num} \rightarrow \text{digits optional_fraction optional_exponent}$

正则表达式的扩展

- 基本运算符：并 连接 闭包
- 扩展运算符
 - 一个或多个： r^+ ，等价于 rr^*
 - 零个或一个： $r?$ ，等价于 $\varepsilon | r$
 - 字符类 $[abc]$ 等价于 $a|b|c$, $[a-z]$ 等价于 $a|b|\dots|z$
- 正则表达式是一种描述手段，通常用来描述程序语言的词法符号

正则表达式的扩展

表达式	匹配	例子
<code>c</code>	单个非运算符字符 <code>c</code>	<code>a</code>
<code>\c</code>	字符 <code>c</code> 的字面值	<code>*</code>
<code>"s"</code>	串 <code>s</code> 的字面值	<code>"**"</code>
<code>.</code>	除换行以外的任何字符	<code>a.*b</code>
<code>^</code>	一行的开始	<code>^abc</code>
<code>\$</code>	行的结尾	<code>abc\$</code>
<code>[s]</code>	字符串 <code>s</code> 中的任何一个字符	<code>[abc]</code>
<code>[^s]</code>	不在串 <code>s</code> 中的任何一个字符	<code>[^abc]</code>
<code>r*</code>	由和 <code>r</code> 匹配的零个或多个串连接成的串	<code>a*</code>
<code>r⁺</code>	由和 <code>r</code> 匹配的一个或多个串连接成的串	<code>a⁺</code>
<code>r?</code>	零个或一个 <code>r</code>	<code>a?</code>
<code>r{m,n}</code>	最少 <code>m</code> 个，最多 <code>n</code> 个 <code>r</code> 的连接	<code>a{1,5}</code>
<code>r₁r₂</code>	<code>r₁</code> 后加上 <code>r₂</code>	<code>ab</code>
<code>r₁ r₂</code>	<code>r₁</code> 或 <code>r₂</code>	<code>a b</code>
<code>(r)</code>	与 <code>r</code> 相同	<code>(a b)</code>
<code>r₁/r₂</code>	后面跟有 <code>r₂</code> 时的 <code>r₁</code>	<code>abc/123</code>

正则表达式课堂练习

- 课堂小问题：

- 以字母或下划线开头，后续是字母、数字或下划线的标识符
- 合法的十进制整数，不能有前导零（0 本身除外）
- 字母表 $\{a,b\}$ 上以 aa 结尾的所有字符串
- 能被 4 整除的二进制数

正则表达式课堂练习

- 课堂小问题：

- 以字母或下划线开头，后续是字母、数字或下划线的标识符

- $[A-Za-z_][A-Za-z0-9_]^*$

- 合法的十进制整数，不能有前导零（0 本身除外）

- $0|[1-9][0-9]^*$

- 字母表 $\{a,b\}$ 上以 aa 结尾的所有字符串

- $(a|b)^*aa$

- 能被 4 整除的二进制数

- $[01]^*00$

正则表达式课堂练习（较难）

- 有一点为一个简化的整数常量（十进制或十六进制）编写正则表达式，满足以下要求：
 - 允许可选的符号位：+ 或 -（最多一个，且只能在最前）
 - 十进制形式：
 - ❑ 不能有多余前导零：0 本身可以，但非零开头时后面可跟数字（例如 0、7、42、123 都合法；012 不合法）
 - 十六进制形式：
 - ❑ 以 0x 或 0X 开头
 - ❑ 后接至少一位十六进制数字（0-9, a-f, A-F）
 - ❑ 十六进制部分允许前导零（例如 0x0、0x00ff 合法）
 - 匹配必须覆盖整个字符串（不允许前后多余字符）

正则表达式课堂练习（较难）

- 有一点为一个简化的整数常量（十进制或十六进制）编写正则表达式，满足以下要求：

- 允许可选的符号位：+ 或 -（最多一个，且只能在最前）

- 十进制形式：

- ❑ 不能有多余前导零：0 本身可以，但非零开头时后面可跟数字（例如 0、7、42、123 都合法；012 不合法）

- 十六进制形式：

- ❑ 以 0x 或 0X 开头

- ❑ 后接至少一位十六进制数字（0-9, a-f, A-F）

- ❑ 十六进制部分允许前导零（例如 0x0、0x00ff 合法）

- 匹配必须覆盖整个字符串（不允许前后多余字符）

`^[+-]?(?:0|[1-9][0-9]*|0[xX][0-9A-Fa-f]+)$`

- ^ 和 \$ 保证整串匹配。
- 可选符号位用 [+-]?。
- 十进制分支拆成 0 与非零开头两种：0 单独一个；[1-9][0-9]* 避免前导零。
- 十六进制分支以 0[xX] 前缀开始，后接至少一位十六进制数字 [0-9A-Fa-f]+。
- 放在一个非捕获分组(?: ...) 中，以便整体与可选符号位结合。

词法单元的识别

词素



词法单元

第二步：如何识别词法单元？

Implementation

词法单元的识别

词素



词法单元

第二步：如何识别词法单元？

Implementation

状态转换图、有限自动机
识别一个串是否属于某个正则集

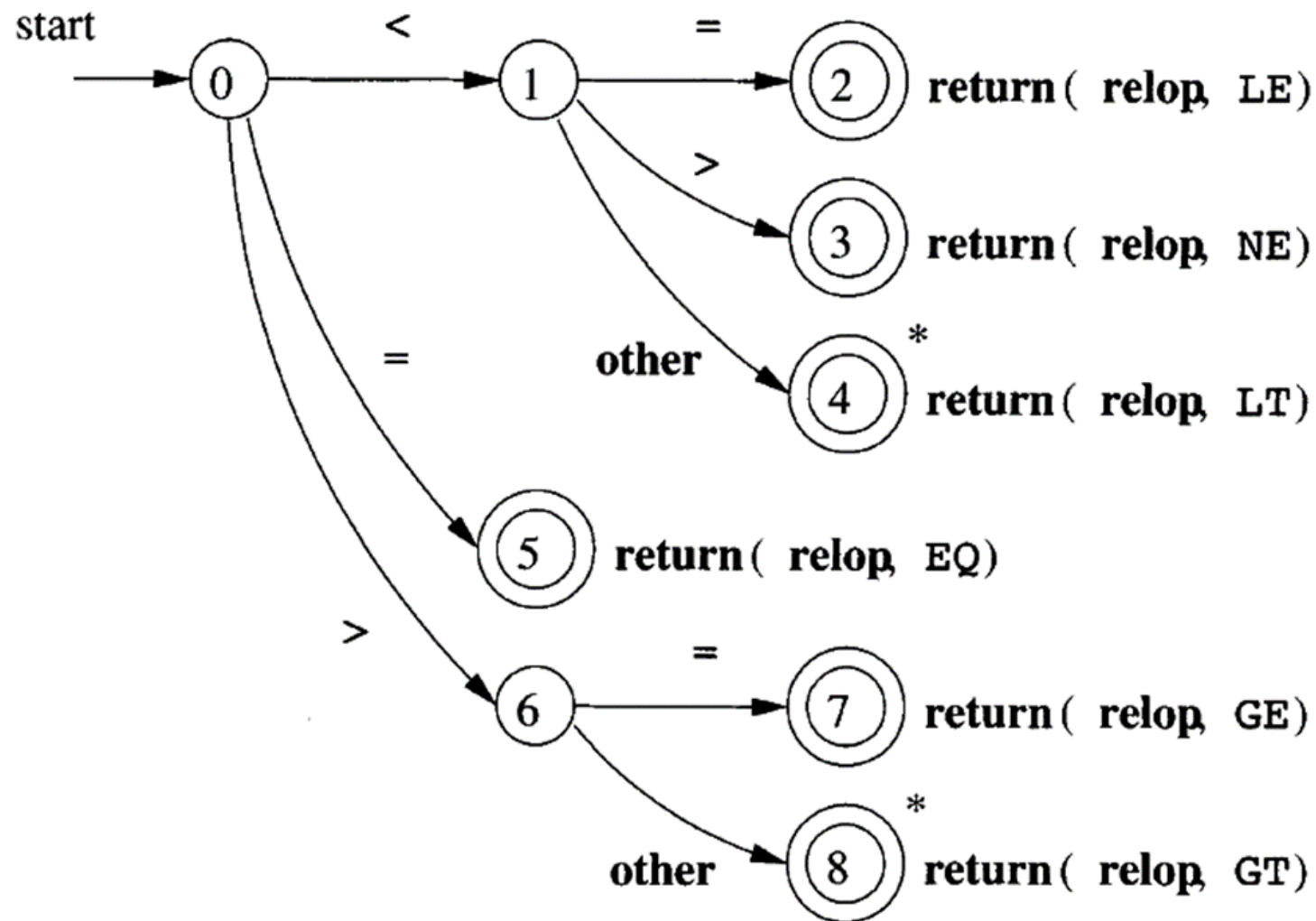
词法单元的识别

- 词法分析器要求能够检查输入字符串，在前缀中找出和某个模式匹配的词素
 - 首先通过正则定义来描述各种词法单元的模式
 - 定义ws→(blank | tab | newline)+来消除空白
 - 词法分析器识别到这个模式时，不返回词法单元，继续识别其它模式

词法单元的识别

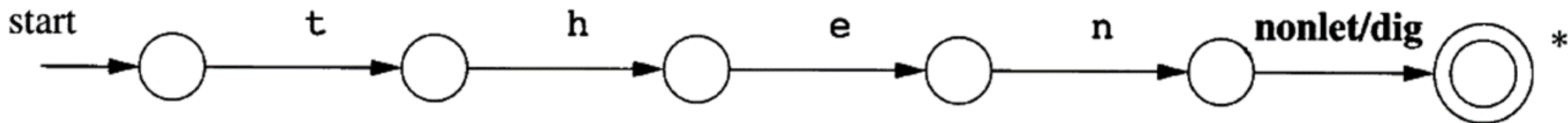
- 状态转换图是词法分析器的重要组成部分之一
- 可以将正则表达式转换成状态转换图
- 状态转换图(transition diagram)
 - 状态(state): 表示在识别词素的过程中可能出现的情况
 - 状态看作是已处理部分的总结
 - 某些状态为接受状态或最终状态, 表明已经找到词素
 - 加上*的接受状态表示最后读入的符号不在词素中
 - 开始状态(初始状态): 用start边表示
 - 边(edge): 从一个状态指向另一个状态; 边的标号是一个或者多个符号
 - 如果当前符号为s, 下一个输入符号为a, 就沿着从s离开, 标号为a的边到达下一个状态

词法单元的认可



词法单元的识别

- 在很多程序设计语言中，保留字也符合标识符的模式，识别标识符的状态转换图也会识别保留字
- 解决方法
 - 在符号表中预先填写保留字，并指明它们不是普通标识符
 - 为关键字/保留字建立单独的状态转换图。并设定保留字的优先级高于标识符
- 词法分析器通常遵循“**最长匹配优先**”和“**规则优先级**”两条原则



假想的关键字 then 的状态转换图

词法单元的识别

- 最长匹配原则（maximal munch）：
 - 定义：从当前位置开始，尽可能读入最多字符，直到再多读就无法匹配任何规则为止，此时回退到最后一个可匹配的位置并产出该词法单元（token）
 - 实现机制：
 - 在扫描时记录“最近一次到达某个规则的接受态”的位置与对应规则
 - 继续读字符直到无法前进；若失败则回溯到最近的接受位置并输出对应 token，然后从那里继续
 - 好处：避免过早截断。例如读取 abc123 时，标识符规则 `[A-Za-z_][A-Za-z0-9_]*` 能匹配整串，而不是只匹配 abc

词法单元的识别

- 最长匹配原则（maximal munch）运行示例：

- 正则表达式集：

- KEYWORDS: "if"、"else"、"return"

- IDENT = [A-Za-z_][A-Za-z0-9_]*

- INT = [0-9]+

- WS = [\t\r\n]+（跳过）

- 输入：ifcount return1

- 扫描步骤：

- 从 i 开始：i → if → ifc → ifco → ... → ifcount

- 说明：虽然子串 “if” 是关键字，但继续读仍能匹配 IDENT，更长，因此不会在 “if” 处停。

- 产出：IDENT("ifcount")

-

词法单元的识别

- 规则优先级

- 当多个规则在同一位置、同一长度上都能匹配时，采用“优先级（规则顺序）”决定。大多数词法工具（Lex/Flex/ANTLR/JavaCC 等）默认“**先出现的规则优先**”
- 这允许我们把“关键字”与“标识符”都写成正则，但先写关键字规则，这样关键字会被优先产出，而不会落入标识符

词法单元的识别

- 规则优先级运行示例1:

- 规则(**注意顺序**):

- "if"

- [A-Za-z_][A-Za-z0-9_]*

- 输入: if(

- 分析: 在读取到 "(" 之前, "if" 与标识符 "if" 同长匹配; 按顺序优先, 产出 IF token, 然后下一次匹配 "("

词法单元的识别

- 规则优先级运行示例2:

- 规则(**注意顺序**):

- "==="

- "=="

- "="

- 输入: ===

- 分析: 在最长匹配下本就会选到 “===”; 若实现仅到等长冲突再比较, 顺序也保证选择 “===”。
原则相辅相成, 确保拿到预期的更长或更特定的 token。

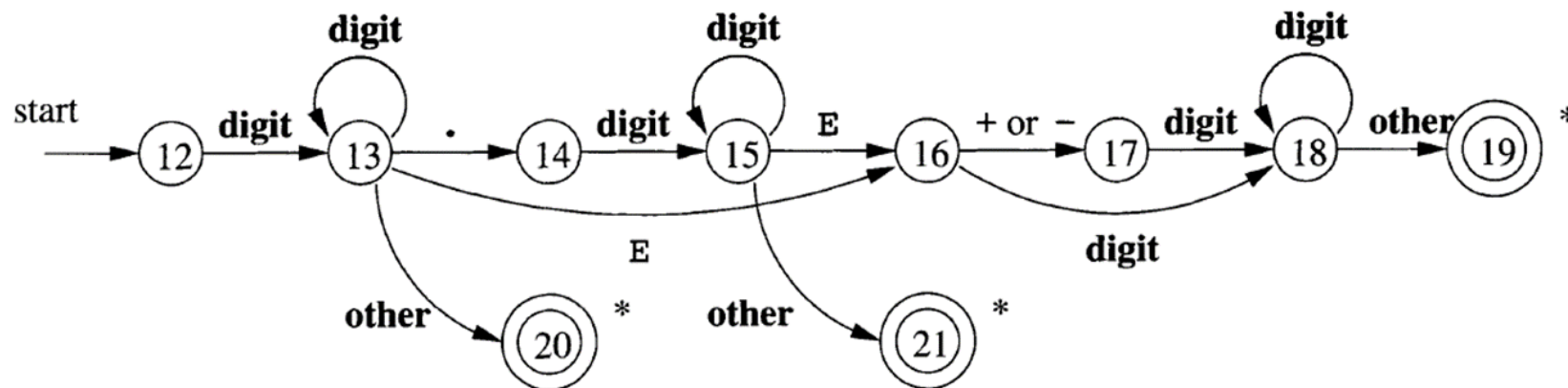
词法单元的识别

- 正则表达式用以描述词法单元匹配的模式
- 状态转换图用以支持实现词法单元的识别
- 那么，如何把正则表达式变化为状态装换图？
- 示例：

digit $\rightarrow$ [0-9]

digits $\rightarrow$ digit+

number $\rightarrow$ digits (. digits)? (E [+ -]? digits)?



有穷自动机的定义

- 本质上等价于状态转换图
- 区别在于：
 - 自动机是识别器，对每个输入串回答yes or no
- 分为两类
 - 非确定有穷自动机（Nondeterministic Finite Automaton, NFA）
 - 确定有穷自动机（Deterministic Finite Automaton, DFA）

有穷自动机

- 不确定 vs. 确定
- 不同：
 - NFA: 一个符号标记离开同一状态的多条边
 - DFA: 对于每个状态和字母表中的每个字符，有且仅有一条离开该状态、以该符号为标号的边
 - NFA: 可以有边的标号是 ϵ
 - DFA: 没有标记为 ϵ 的边
- 相同：都可以识别正则语言，两者之间存在等价性

非确定有穷自动机

- 非确定有穷自动机 (NFA)
- NFA由以下几部分组成
 - 一个有穷的状态集合 S
 - 一个输入符号集合 Σ (input alphabet)
 - 转换函数 (transition function) 对于每个状态和 $\Sigma \cup \{\varepsilon\}$ 中的符号, 给出相应的后继状态集合
 - 一个状态 s_0 被指定为开始状态/初始状态
 - S 的一个子集 F 被指定为接受状态

非确定有穷自动机

- NFA的一个例子

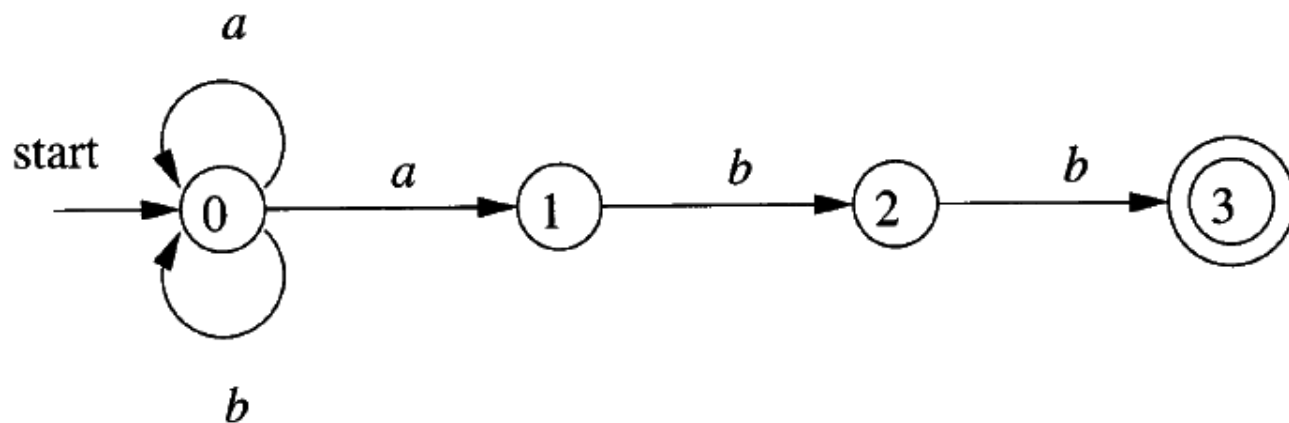
- 状态集合 $S = \{0, 1, 2, 3\}$

- 开始状态 0

- 接受状态集合 $\{3\}$

- 转换函数:

□ $(0, a) \rightarrow \{0, 1\}$ $(0, b) \rightarrow \{0\}$ $(1, b) \rightarrow 2$ $(2, b) \rightarrow 3$



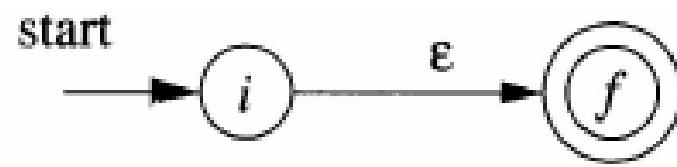
非确定有穷自动机

- 正则表达式到NFA
- 输入：字母表 Σ 上的一个正则表达式 r
- 输出：一个接受 $L(r)$ 的NFA N
- 基本思想
 - 根据正则表达式的递归定义，按照正则表达式的结构递归地构造出相应的NFA
 - 算法分成两个部分：
 - 基本规则处理 ϵ 和单符号的情况
 - 对于每个正则表达式的运算，建立构造相应NFA的方法

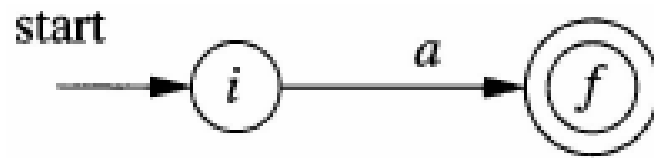
非确定有穷自动机

- 正则表达式到NFA
- 基本规则：

➤ 表达式 ϵ ,



➤ 表达式 a ,

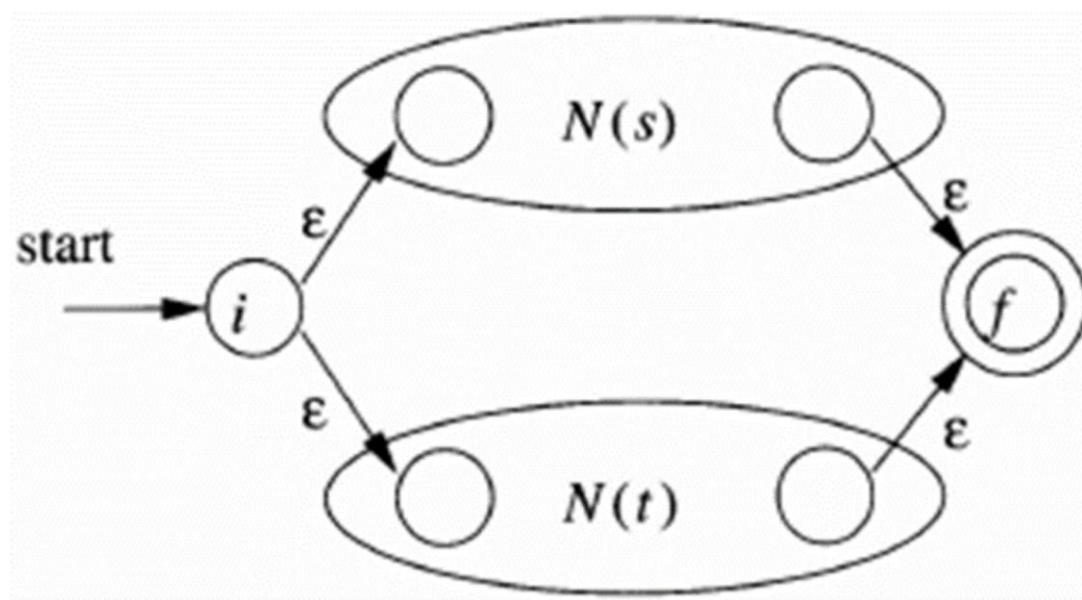


非确定有穷自动机

- 正则表达式到NFA
- 归纳规则
 - 正则表达式 s 和 t 的NFA分别是 $N(s)$ 和 $N(t)$
 - $r=s|t$, r 的NFA $N(r)$

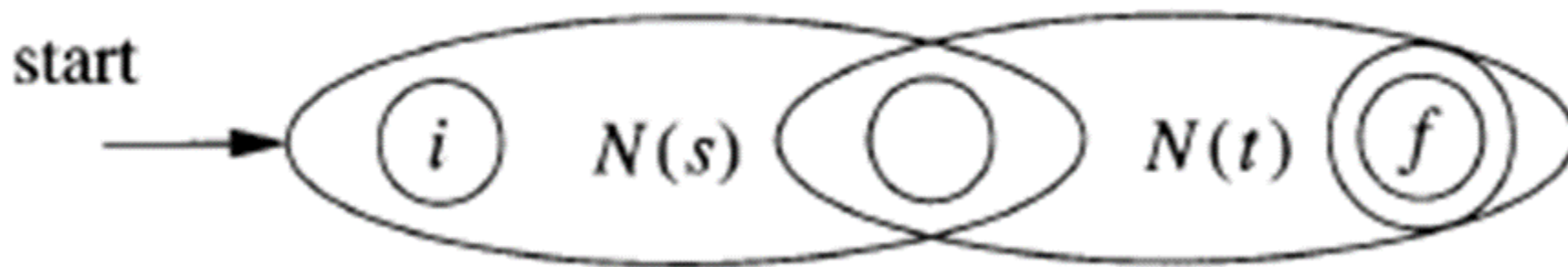
非确定有穷自动机

- 正则表达式到NFA
- 归纳规则
 - 正则表达式 s 和 t 的NFA分别是 $N(s)$ 和 $N(t)$
 - $r=s|t$, r 的NFA $N(r)$



非确定有穷自动机

- 正则表达式到NFA
- 归纳规则
 - 正则表达式 $r=st$, $N(r)$



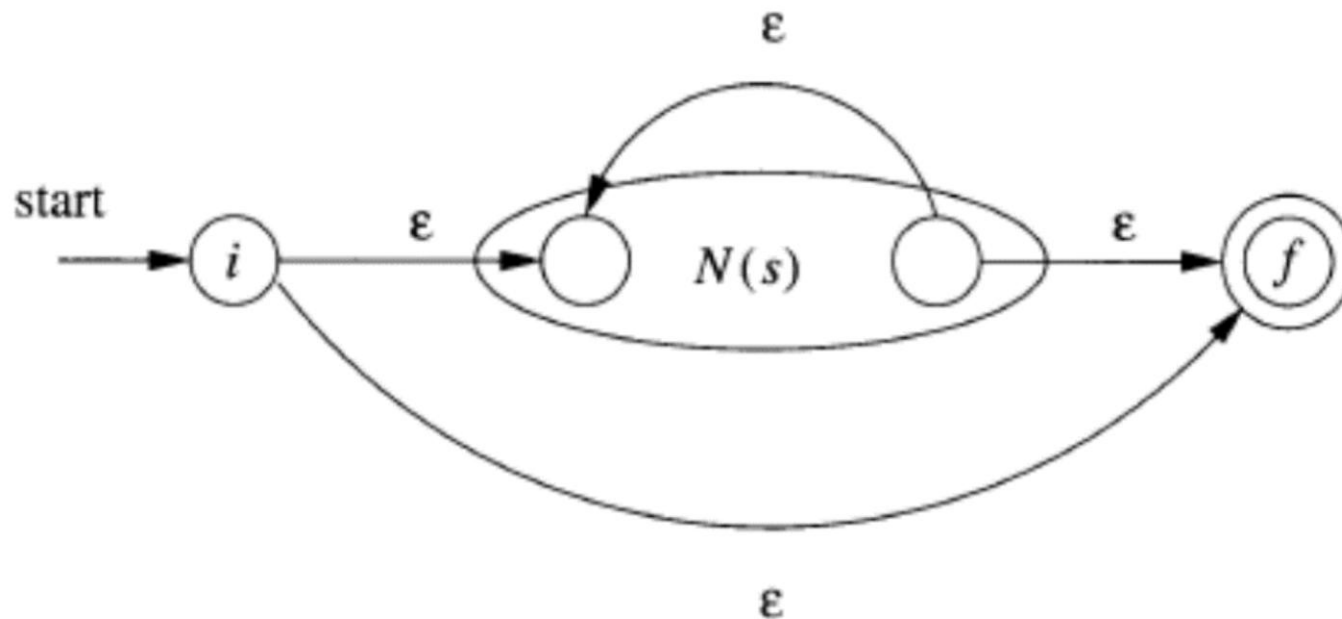
非确定有穷自动机

- 正则表达式到NFA
- 归纳规则
 - 正则表达式 $r=s^*$, $N(r)$

➤ $r=(s)$, $N(r)=N(s)$

非确定有穷自动机

- 正则表达式到NFA
- 归纳规则
 - 正则表达式 $r=s^*$, $N(r)$



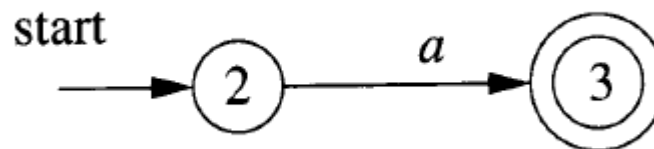
➤ $r=(s)$, $N(r)=N(s)$

非确定有穷自动机

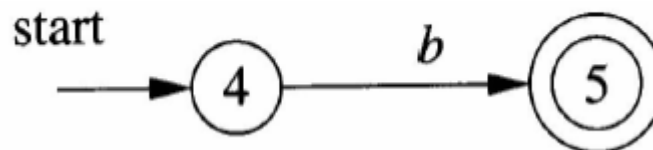
- 正则表达式到NFA的例子
- 正则表达式 $(a|b)^*abb$
- 第一个a对应的NFA
- 第一个b对应的NFA

非确定有穷自动机

- 正则表达式到NFA的例子
- 正则表达式 $(a|b)^*abb$
- 第一个a对应的NFA



- 第一个b对应的NFA

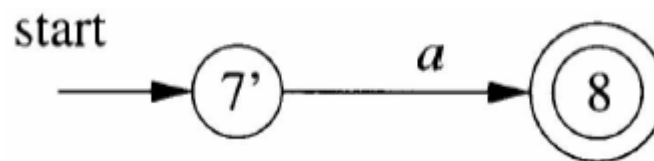
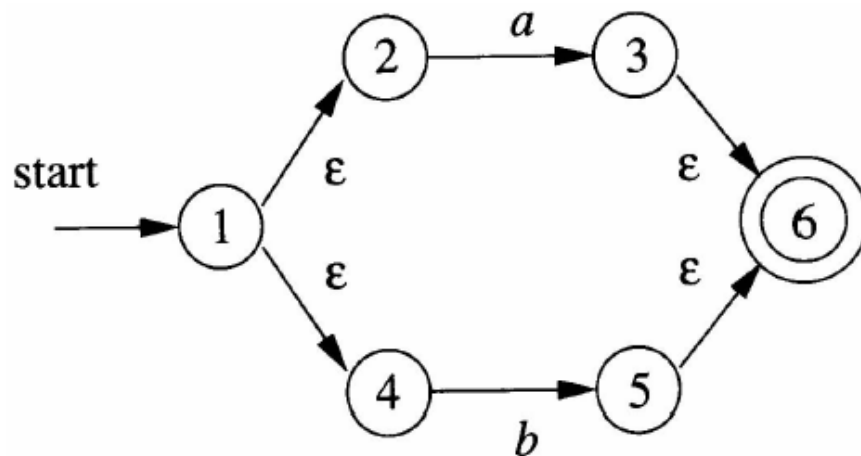


非确定有穷自动机

- 正则表达式到NFA的例子
- 正则表达式 $(a|b)^*abb$
- $(a|b)$ 的NFA
- 第二个a的NFA

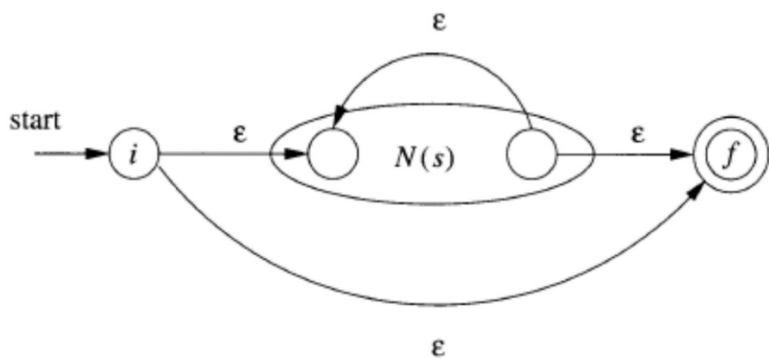
非确定有穷自动机

- 正则表达式到NFA的例子
- 正则表达式 $(a|b)^*abb$
- $(a|b)$ 的NFA
- 第二个a的NFA



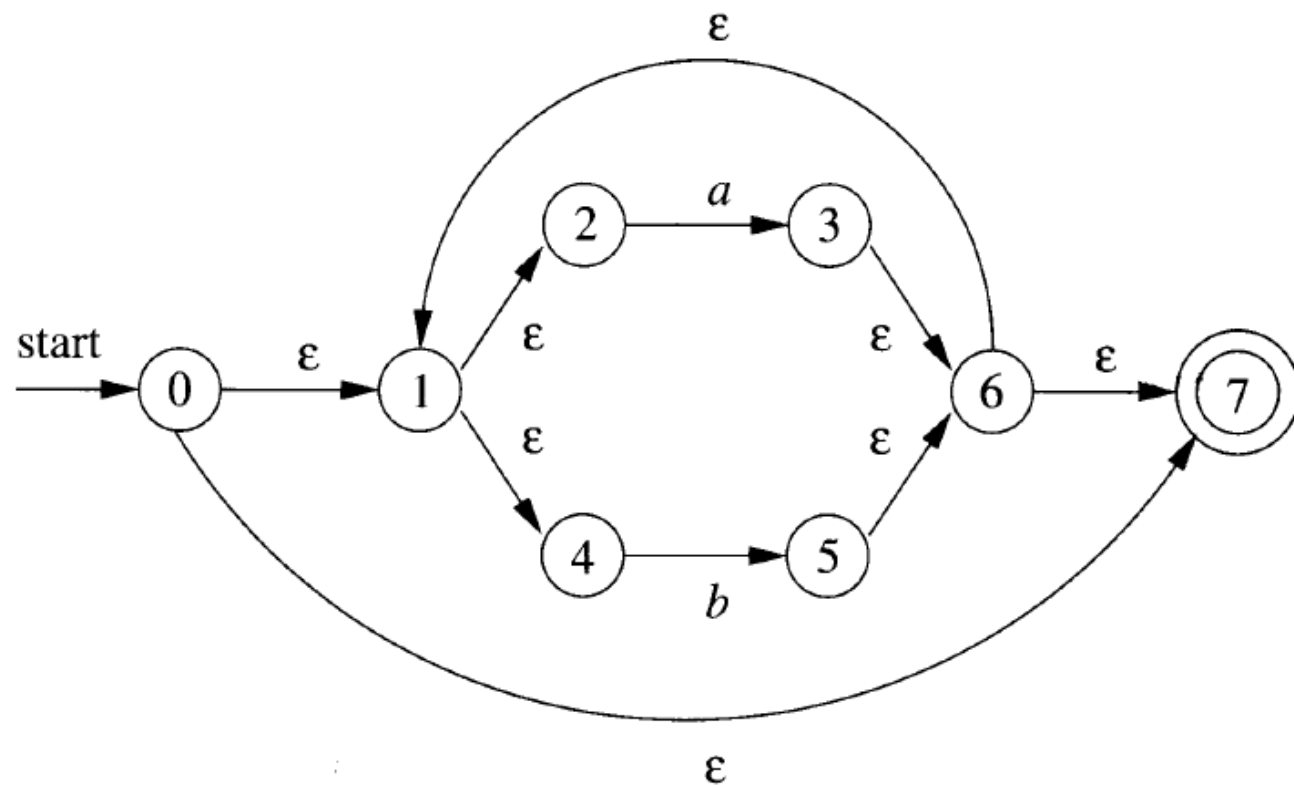
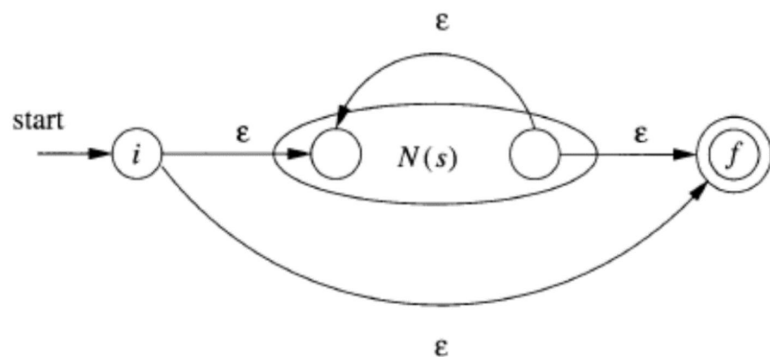
非确定有穷自动机

- 正则表达式到NFA的例子
- 正则表达式 $(a|b)^*abb$
- $(a|b)^*$ 的NFA



非确定有穷自动机

- 正则表达式到NFA的例子
- 正则表达式 $(a|b)^*abb$
- $(a|b)^*$ 的NFA



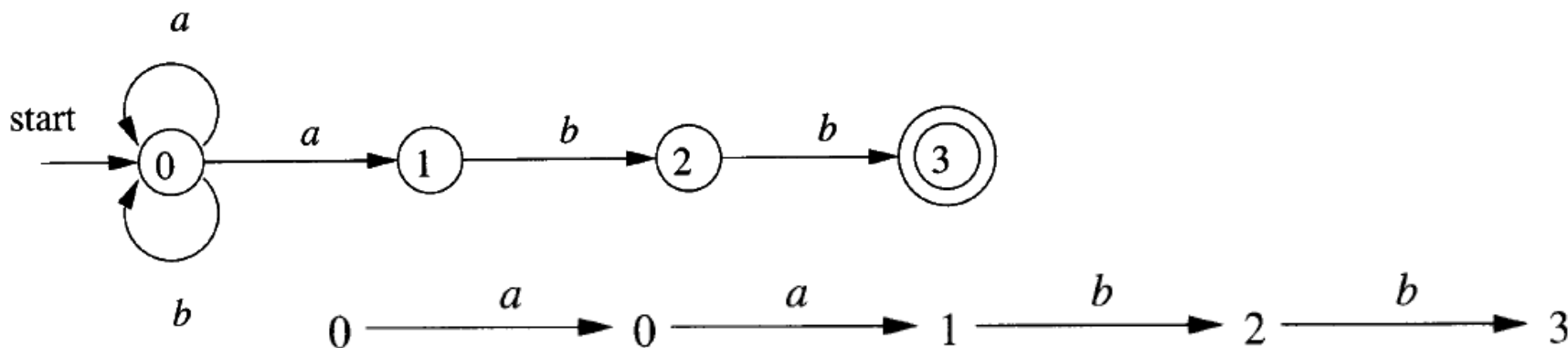
非确定有穷自动机

- NFA可以表示为一个转换表
 - 表的各行对应于状态
 - 各列对应于输入符号和 ϵ
 - 表中的元素表示给定状态在给定输入下的后继状态

状态	a	b	ϵ
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

非确定有穷自动机

- 一个NFA接受输入字符串 x ，当且仅当对应的转换图中存在一条从开始状态到某个接受状态的路径，使得该路径中各条边上的标号组成符号串 x （路径中可能包含 ε 边）
- 下图对应的NFA能够接受aabb

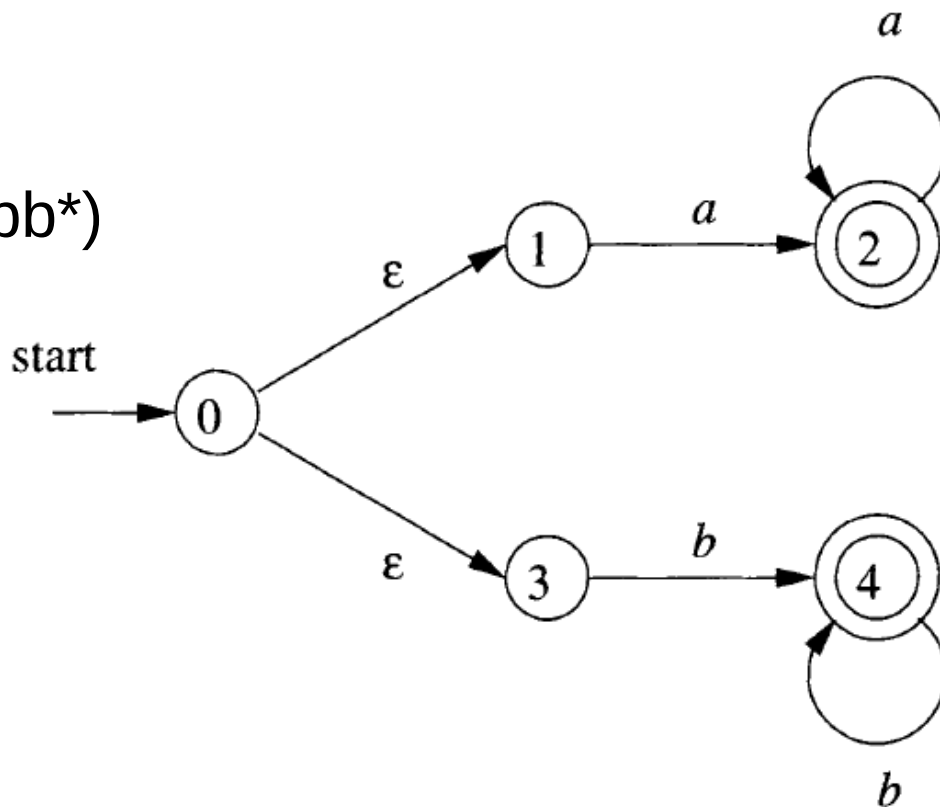


注意：只要存在从开始状态到接受状态的路径，符号串就认为被NFA接受

非确定有穷自动机

- 由一个NFA A 定义（接受）的语言是从开始状态到某个接受状态的所有路径上的符号串集合，称为 $L(A)$

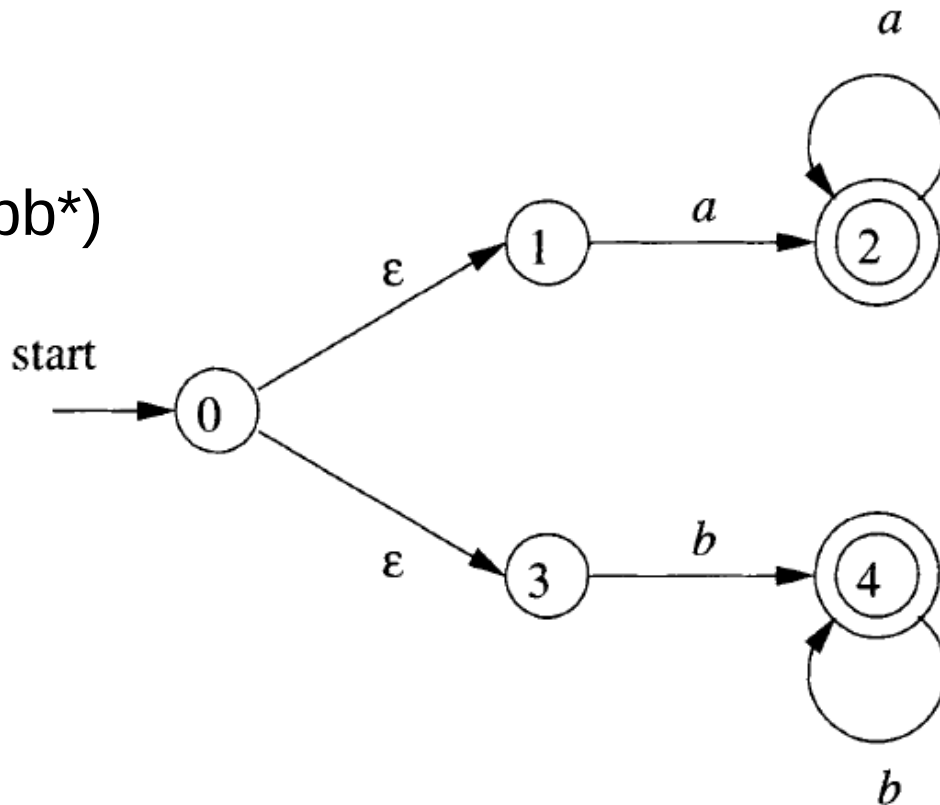
相应的语言： $L(aa^*|bb^*)$



非确定有穷自动机

- 由一个NFA A 定义（接受）的语言是从开始状态到某个接受状态的所有路径上的符号串集合，称为 $L(A)$

相应的语言： $L(aa^*|bb^*)$



确定有穷自动机

- 确定有穷自动机 (DFA)
- 一个NFA被称为DFA，如果
 - 没有 ε 之上的转换动作
 - 对于每个状态 s 和每个输入符号 a ，有且只有一条标号为 a 的边
- 可以高效判断一个串能否被一个DFA接受
- 每个NFA都有一个等价的DFA，即它们接受同样的语言

确定有穷自动机

- 假设输入符号就是字符串中的符号；
- Nextchar读入下一个字符（符号）
- move给出了离开s，标号为c的边的目标状态

```
s = s0;  
c = nextChar();  
while ( c != eof ) {  
    s = move(s, c);  
    c = nextChar();  
}  
if ( s 在 F 中 ) return "yes";  
else return "no";
```

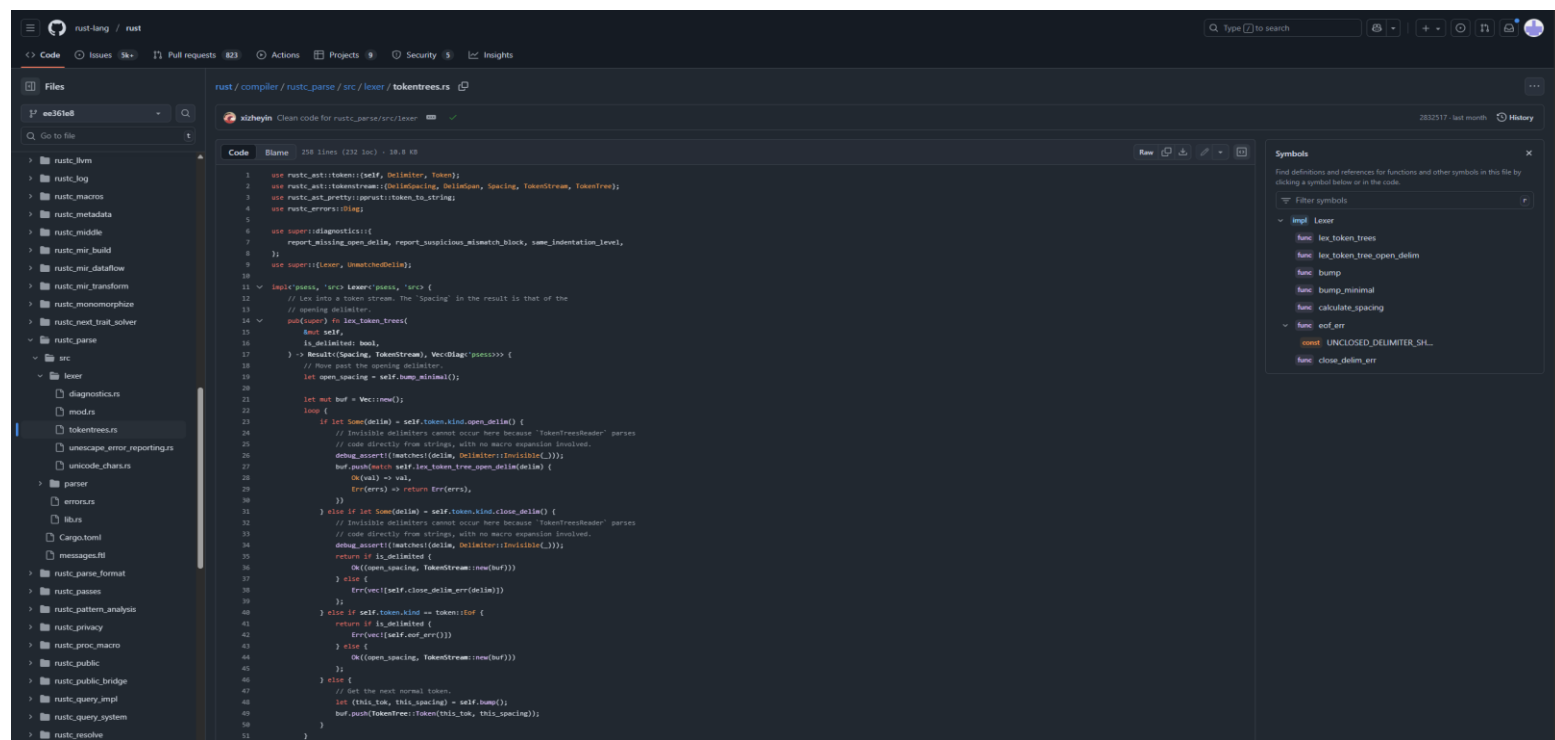
确定有穷自动机

- 假设输入符号就是字符串中的符号；
- Nextchar读入下一个字符（符号）
- move给出了离开s，标号为c的边的目标状态

```

s = s0;
c = nextChar();
while ( c != eof ) {
    s = move(s, c);
    c = nextChar();
}
if ( s在F中 ) return "yes";
else return "no";

```

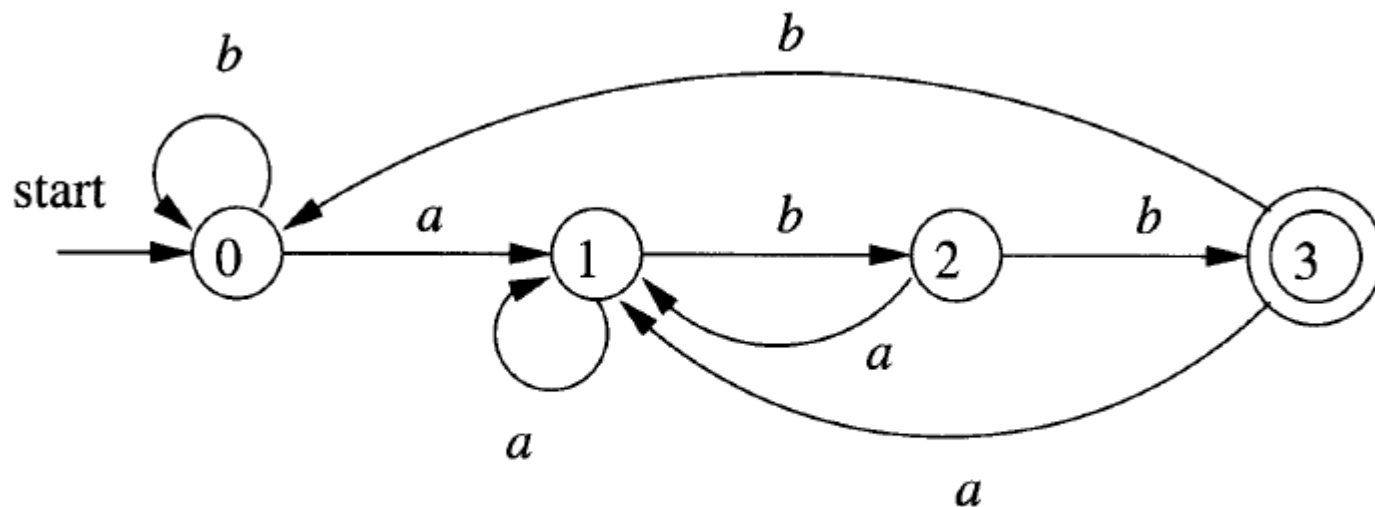


https://github.com/rust-lang/rust/blob/ee361e8fca1c30e13e7a31cc82b64c045339d3a8/compiler/rustc_parse/src/lexer/tokentrees.rs

确定有穷自动机

- DFA 运行的例子

➤ 假设输入为ababb, 那么进入的状态序列为 0,1,2,1,2,3, 返回yes



接受 $(alb)^*abb$ 的DFA

词素



词法单元

我们已有的工具：

1. RE
2. NFA
3. DFA

确定有穷自动机

- 正则表达式到自动机
- 正则表达式可以简洁、精确地描述词法单元的模式
- 进行模式匹配时，模拟DFA的执行比模拟NFA更为简单有效
 - ϵ 的模拟，难
 - 不确定性的模拟，难
- 因此，需要将正则表达式转换为DFA
- 步骤：
 - 正则表达式到NFA
 - NFA到DFA

确定有穷自动机

- NFA转换成DFA - 子集构造法
- 对NFA的模拟往往不如对DFA的模拟直接，除非转换花费更多的时间
- 基本思想： DFA每个状态 $\leftrightarrow$ NFA一个状态集
 - “并行地模拟” NFA在遇到一个给定输入串时可能执行的所有动作
 - 构造得到的DFA的每个状态和NFA的状态子集对应
 - DFA读入 $a_1, a_2, \dots, a_n$ 后到达的状态对应于从NFA开始状态出发沿着 $a_1, a_2, \dots, a_n$ 可能到达的状态集合
- 理论上，最坏情况下DFA的状态个数会是NFA状态个数的指数多个。
但是对于大部分应用，NFA和相应的DFA的状态数量大致相同

确定有穷自动机

- 输入：一个NFA N
 - 输出：一个接受相同语言的DFA D
- s 表示 N 中的单个状态， T 代表 N 的一个状态集

操作	描述
$\epsilon\text{-closure}(s)$	能够从NFA的状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合
$\epsilon\text{-closure}(T)$	能够从 T 中某个NFA状态 s 开始只通过 ϵ 转换到达的NFA 状态集合，即 $\cup_{s \in T} \epsilon\text{-closure}(s)$.
$\text{move}(T, a)$	能够从 T 中某个状态 s 出发通过标号为 a 的转换到达的 NFA状态的集合

确定有穷自动机

- 输入：一个NFA N
- 输出：一个接受相同语言的DFA D
 - D 的开始状态是 ϵ -closure(s_0)， D 的接受状态是所有至少包含了 N 的一个接受状态的状态集合。
 - 对NFA的任何状态集合 ϵ -closure(T)的计算
 - 一个图搜索过程

```
一开始,  $\epsilon$ -closure( $s_0$ )是  $Dstates$  中的唯一状态, 且它未加标记;  
while ( 在  $Dstates$  中有一个未标记状态  $T$  ) {  
    给  $T$  加上标记;  
    for ( 每个输入符号  $a$  ) {  
         $U = \epsilon$ -closure( $move(T, a)$ );  
        if (  $U$  不在  $Dstates$  中 )  
            将  $U$  加入到  $Dstates$  中, 且不加标记;  
         $Dtran[T, a] = U$ ;  
    }  
}
```

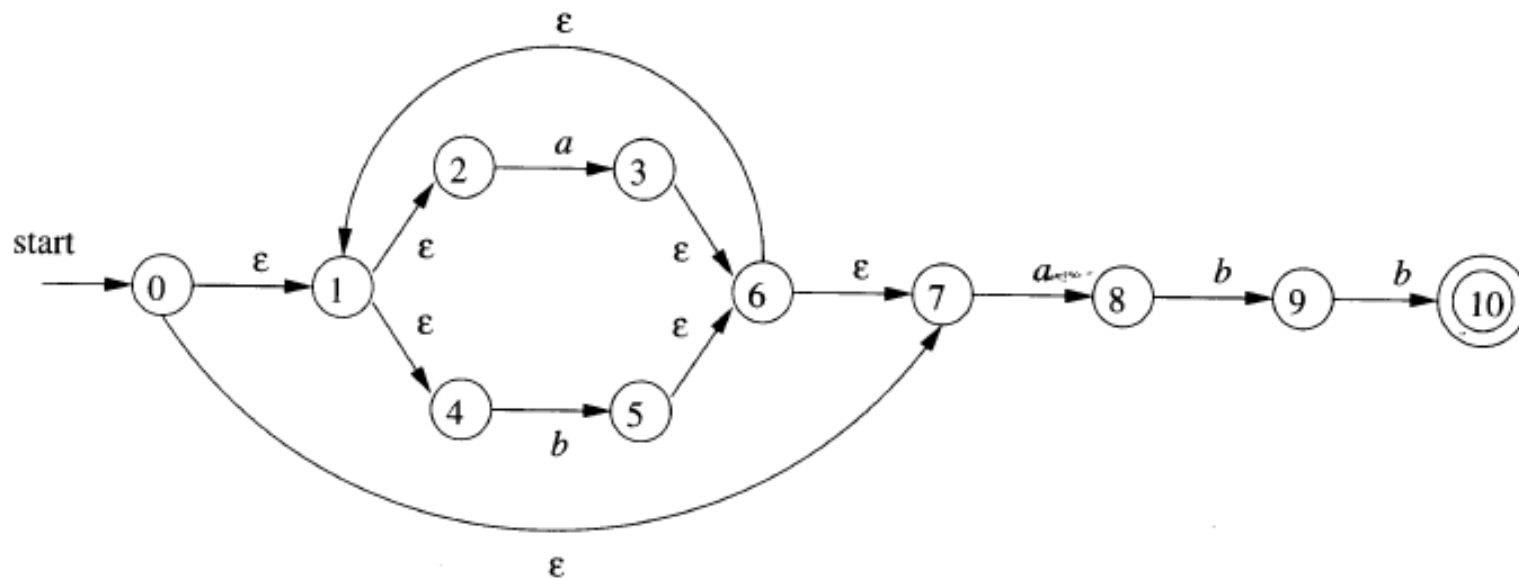
确定有穷自动机

- 输入：一个NFA N
- 输出：一个接受相同语言的DFA D
 - D 的开始状态是 ϵ -closure(s_0)， D 的接受状态是所有至少包含了 N 的一个接受状态的状态集合。
 - 对NFA的任何状态集合 ϵ -closure(T)的计算
 - 一个图搜索过程

```
将 $T$ 的所有状态压入  $stack$  中；  
将  $\epsilon$ -closure( $T$ ) 初始化为  $T$ ；  
while (  $stack$  非空 ) {  
    将栈顶元素  $t$  弹出栈中；  
    for ( 每个满足如下条件的  $u$  : 从  $t$  出发有一个标号为  $\epsilon$  的转换到达状态  $u$  )  
        if (  $u$  不在  $\epsilon$ -closure( $T$ ) 中 ) {  
            将  $u$  加入到  $\epsilon$ -closure( $T$ ) 中；  
            将  $u$  压入栈中；  
        }  
}
```

确定有穷自动机

- A: $=\epsilon\text{-closure}(0)=\{0,1,2,4,7\}$
- B: $\text{Dtran}[A,a]=\epsilon\text{-closure}(\text{move}(A,a))=\epsilon\text{-closure}(\{3,8\})=\{1,2,3,4,6,7,8\}$
- C: $\text{Dtran}[A,b]=\epsilon\text{-closure}(\text{move}(A,b))=\epsilon\text{-closure}(\{5\})=\{1,2,4,5,6,7\}$
- D: $\text{Dtran}[B,b]=\epsilon\text{-closure}(\text{move}(B,b))=\{1,2,4,5,6,7,9\}$
-
- E: $\text{Dtran}[D,b]=\epsilon\text{-closure}(\text{move}(B,b))=\{1,2,4,5,6,7,10\}$

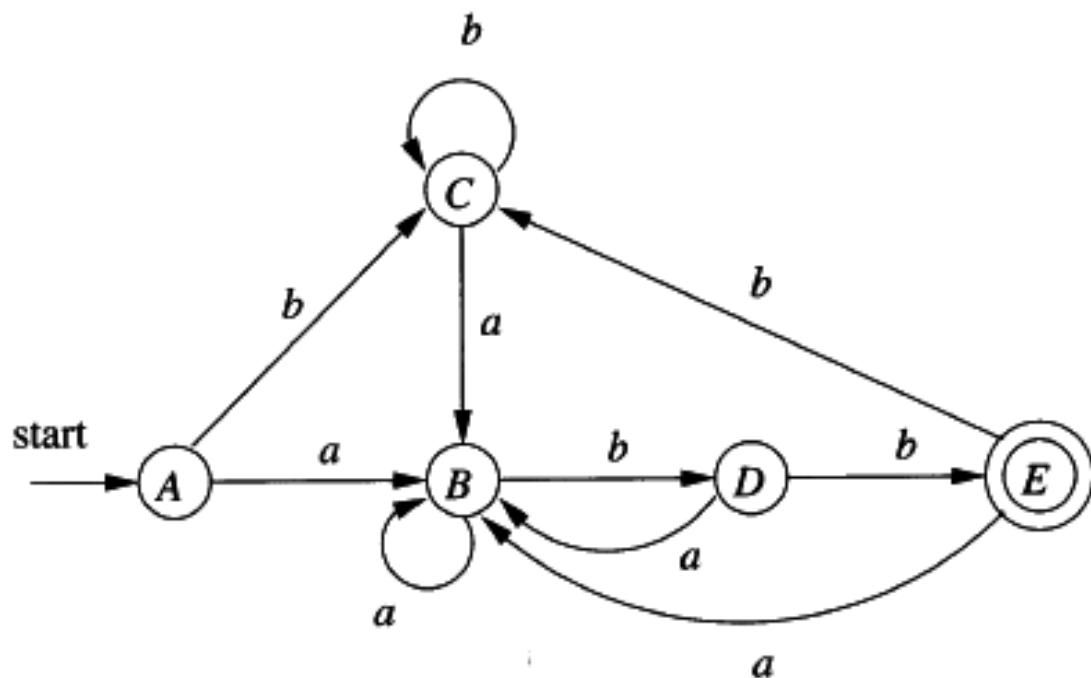


确定有穷自动机

• NFA到DFA转换的示例

- 开始状态：A
- 接受状态：E

NFA 状态	DFA 状态	a	b
{0, 1, 2, 4, 7}	A	B	C
{1, 2, 3, 4, 6, 7, 8}	B	B	D
{1, 2, 4, 5, 6, 7}	C	B	C
{1, 2, 4, 5, 6, 7, 9}	D	B	E
{1, 2, 4, 5, 6, 7, 10}	E	B	C



确定有穷自动机

- 对NFA的运行进行模拟

- 输入：一个以文件结束符eof结尾的输入串 x ，一个NFA N ，其开始状态是 S_0 ，接受状态集为 F ，转换函数为 $move$
- 输出：如果 N 接受 x ，返回yes，否则返回no

```
1)   $S = \epsilon\text{-closure}(s_0);$ 
2)   $c = nextChar();$ 
3)  while (  $c \neq eof$  ) {
4)       $S = \epsilon\text{-closure}(move(S, c));$ 
5)       $c = nextChar();$ 
6)  }
7)  if (  $S \cap F \neq \emptyset$  ) return "yes";
8)  else return "no";
```

确定有穷自动机

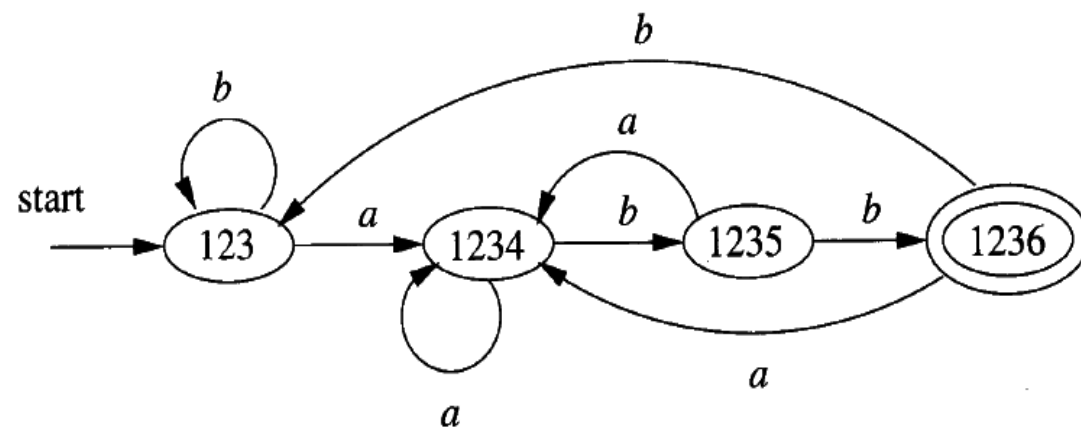
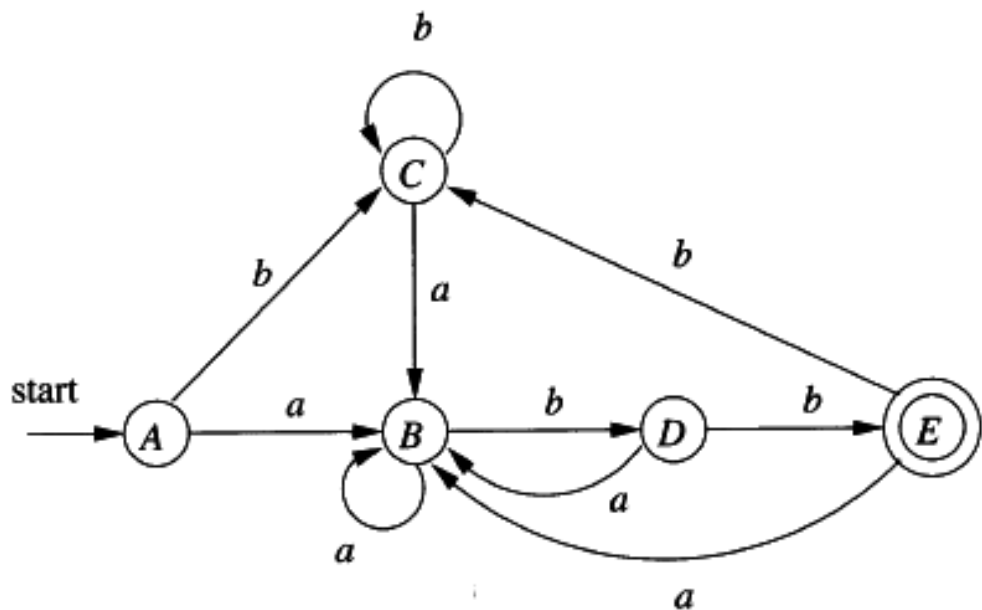
- 对NFA的运行进行模拟

- 输入：一个以文件结束符eof结尾的输入串 x ，一个NFA N ，其开始状态是 S_0 ，接受状态集为 F ，转换函数为 $move$
- 输出：如果 N 接受 x ，返回yes，否则返回no

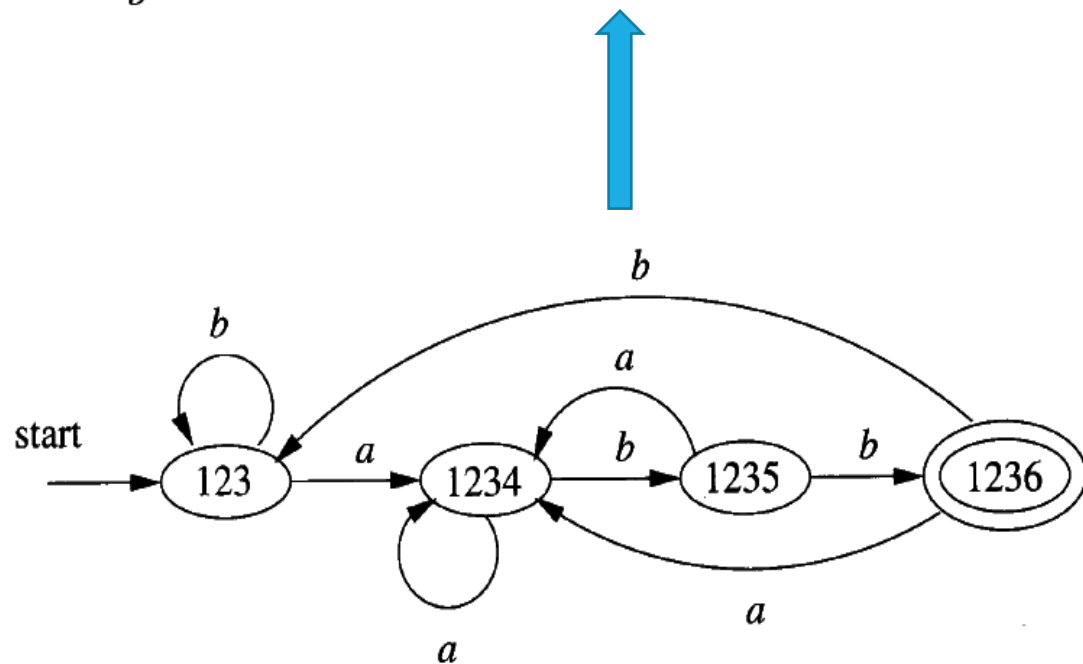
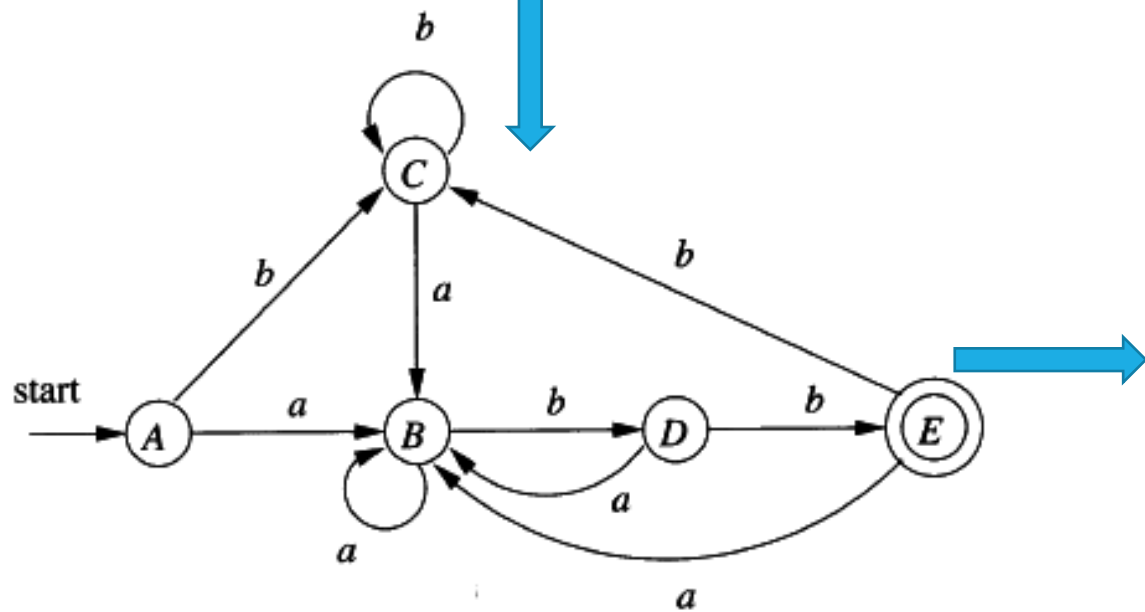
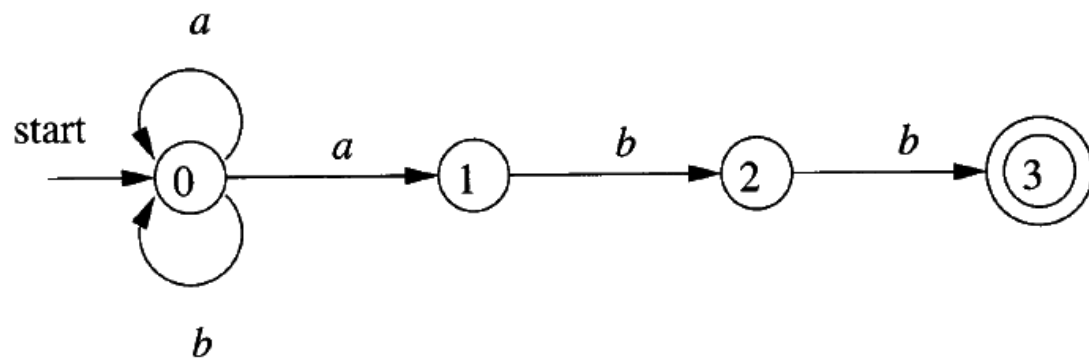
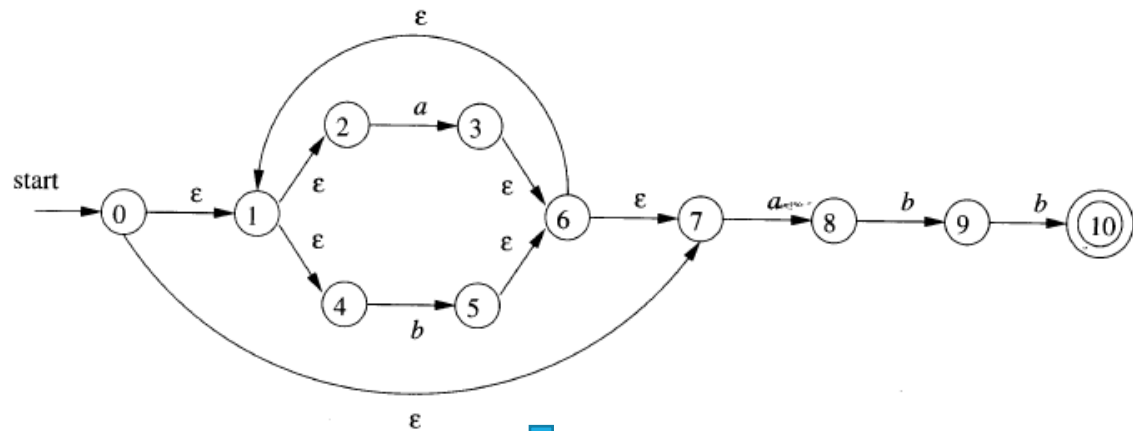
```
1)   $S = \epsilon\text{-closure}(s_0);$   
2)   $c = nextChar();$   
3)  while (  $c \neq eof$  ) {  
4)       $S = \epsilon\text{-closure}(move(S, c));$   
5)       $c = nextChar();$   
6)  }  
7)  if (  $S \cap F \neq \emptyset$  ) return "yes";  
8)  else return "no";
```

确定有穷自动机

- Recap: 前面提到, DFA 的状态数量可能是等价NFA 的指数倍
- DFA化简: 状态数最小化
- 等价的DFA可能具有不同的状态个数
- 任何正则语言都有一个唯一的 (不计同构) 状态数目最少的DFA

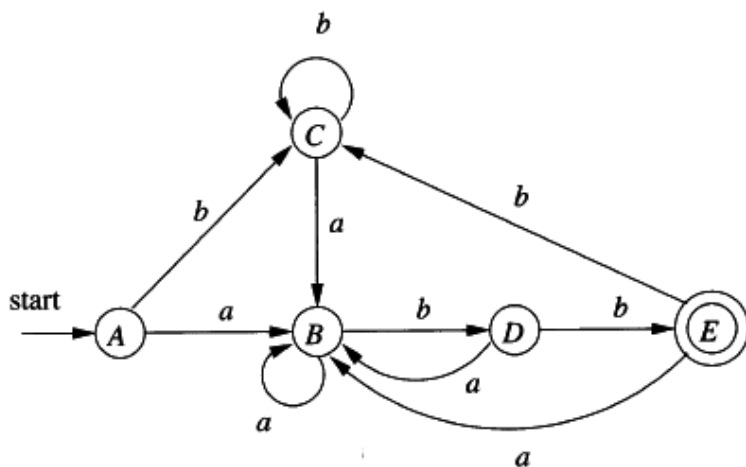


确定有穷自动机



确定有穷自动机

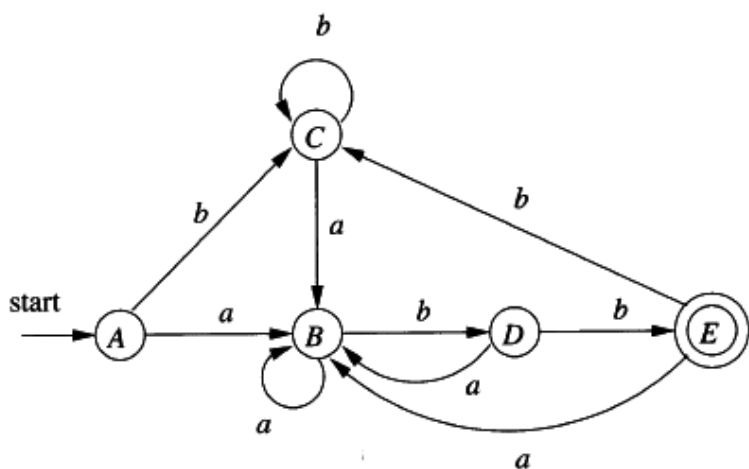
- DFA化简：状态数最小化
- 原理：将一个DFA的状态集合**分划**成多个组，每个组中的各状态之间相互**不可区分**，然后将每个组中的状态**合并**成状态最少DFA的一个状态
- 可区分的定义：
 - 如果分别从状态s和状态t出发，沿着标号为x的路径到达的两个状态只有一个是接受状态，称为x区分状态s和t
 - 如果**存在能够区分s和t的串**，那么它们就是可区分的



空串区分了E和其它状态
bb区分了A和B

确定有穷自动机

- DFA化简：状态数最小化
- 原理：将一个DFA的状态集合**分划**成多个组，每个组中的各状态之间相互**不可区分**，然后将每个组中的状态**合并**成状态最少DFA的一个状态
- 可区分的定义：
 - 如果分别从状态s和状态t出发，沿着标号为x的路径到达的两个状态只有一个是接受状态，称为x区分状态s和t
 - 如果**存在能够区分s和t的串**，那么它们就是可区分的



空串区分了E和其它状态
bb区分了A和B

A和C是可区分的吗？

确定有穷自动机

DFA化简：状态数最小化算法

1. 设置初始分划 $\Pi = \{S-F, F\}$
2. 迭代，不断分划：
for (Π 中的每个元素 G) {
 细分 G ，使得 G 中的 s 、 t 仍然在同一组中 iff
 对任意 a ， s, t 都到达 Π 中的同一组；
 Π_{new} = 将 Π 中的 G 替换为细分得到的小组；
}
3. 如果 $\Pi_{\text{new}} == \Pi$ ，令 $\Pi_{\text{final}} == \Pi$ ，转步骤4；否则 $\Pi == \Pi_{\text{new}}$ ，转步骤2；

确定有穷自动机

DFA化简：状态数最小化算法

4. 在 Π_{final} 的每个组中选择一个状态作代表，作为最小DFA的状态

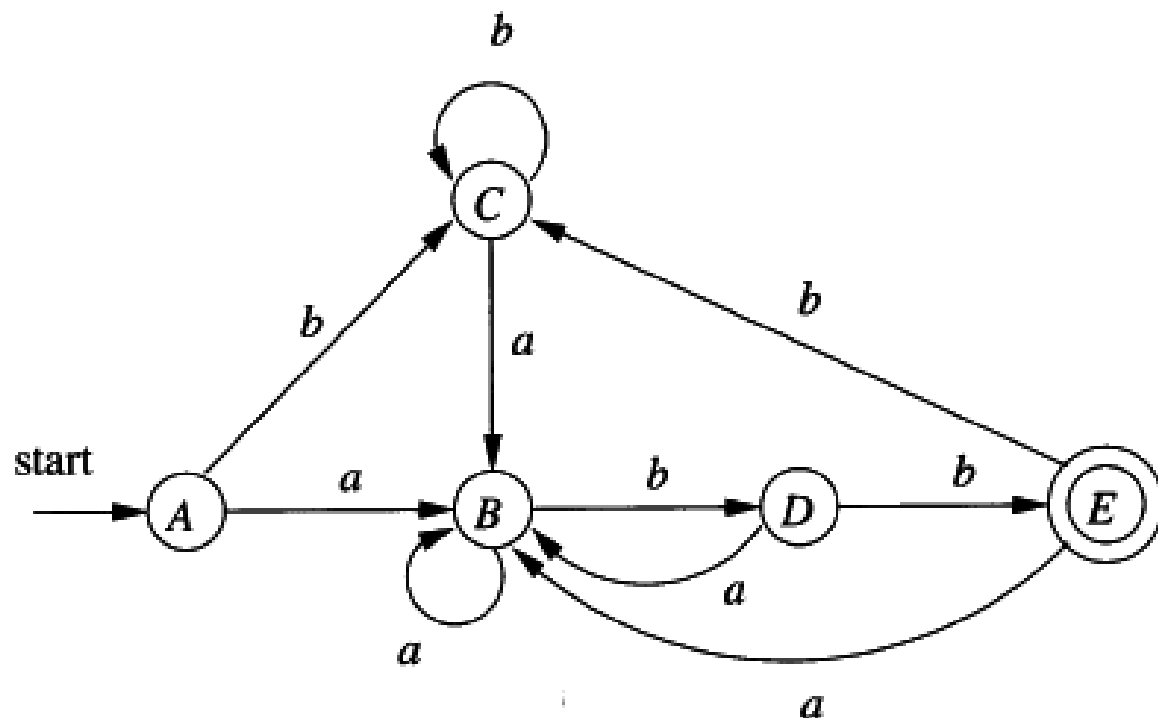
- 开始状态就是中包含原开始状态的组的代表
- 接受状态就是包含了原接受状态的组的代表
- 转化关系构造如下：

□ 如果s是中G的代表，而s在a上的转换到达t，而t所在组的代表为r，那么最小DFA中有从s到r的、在a上的转换

确定有穷自动机

• DFA化简的示例

- 初始分划: $\{A, B, C, D\}$ $\{E\}$
- 处理 $\{A, B, C, D\}$, b 把它细分为 $\{A, B, C\}$ $\{D\}$
- 处理 $\{A, B, C\}$, b 把它细分为 $\{A, C\}$ $\{B\}$
- 分划完毕。选取 A, B, D 和 E 作为代表, 构造得到最小DFA



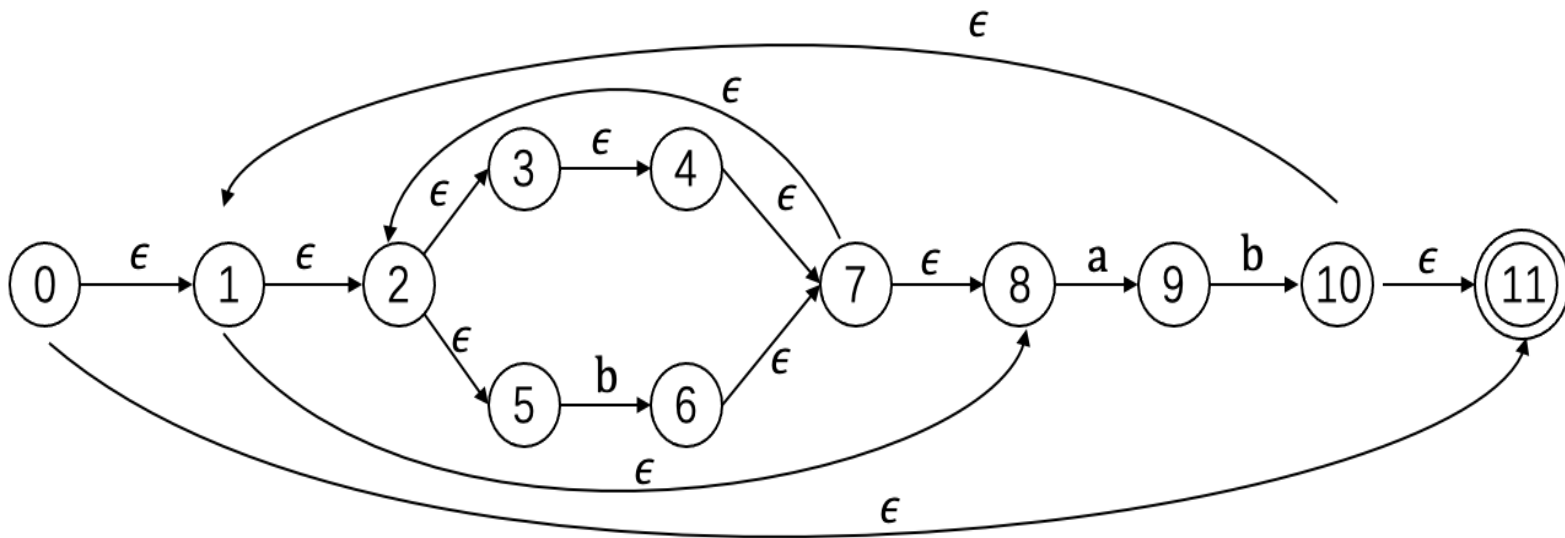
状态	a	b
A	B	A
B	B	D
D	B	E
E	B	A

确定有穷自动机课堂练习（较难）

- 考虑正则表达式 $r = ((\epsilon | b)^* ab)^*$ 。
 - (1) 请使用Thompson 构造法构造等价的NFA
 - (2) 请使用子集构造法构造等价的DFA(注: 请标明状态之间的对应关系)
 - (3) 请将上一步构造的DFA 最小化

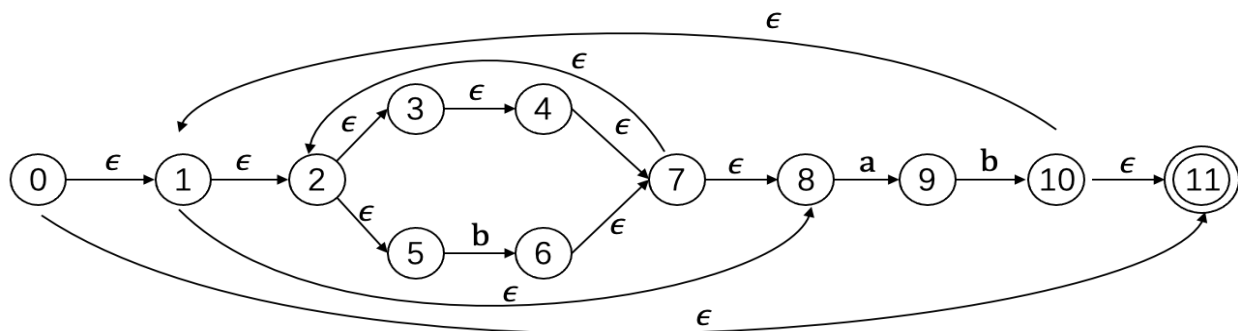
确定有穷自动机课堂练习（较难）

- 考虑正则表达式 $r = ((\epsilon | b)^* ab)^*$ 。
- (1) 请使用Thompson 构造法构造等价的NFA
- (2) 请使用子集构造法构造等价的DFA(注: 请标明状态之间的对应关系)
- (3) 请将上一步构造的DFA 最小化

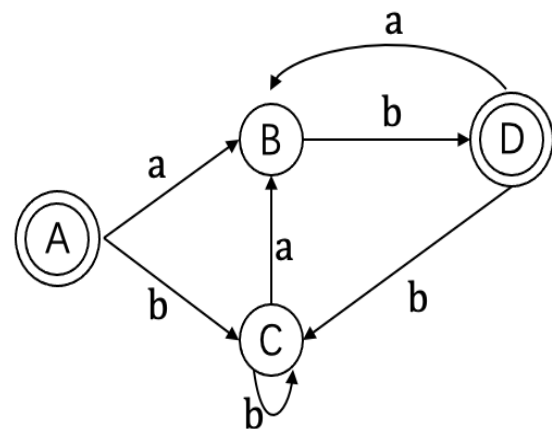


确定有穷自动机课堂练习（较难）

- 考虑正则表达式 $r = ((\epsilon | b)^* ab)^*$ 。
- (1) 请使用Thompson 构造法构造等价的NFA
- (2) 请使用子集构造法构造等价的DFA(注: 请标明状态之间的对应关系)
- (3) 请将上一步构造的DFA 最小化



NFA	DFA	a	b
{0,1,2,3,4,5,7,8,11}	A	B	C
{9}	B	ϕ	D
{2,3,4,5,6,7,8}	C	B	C
{1,2,3,4,5,7,8,10,11}	D	B	C



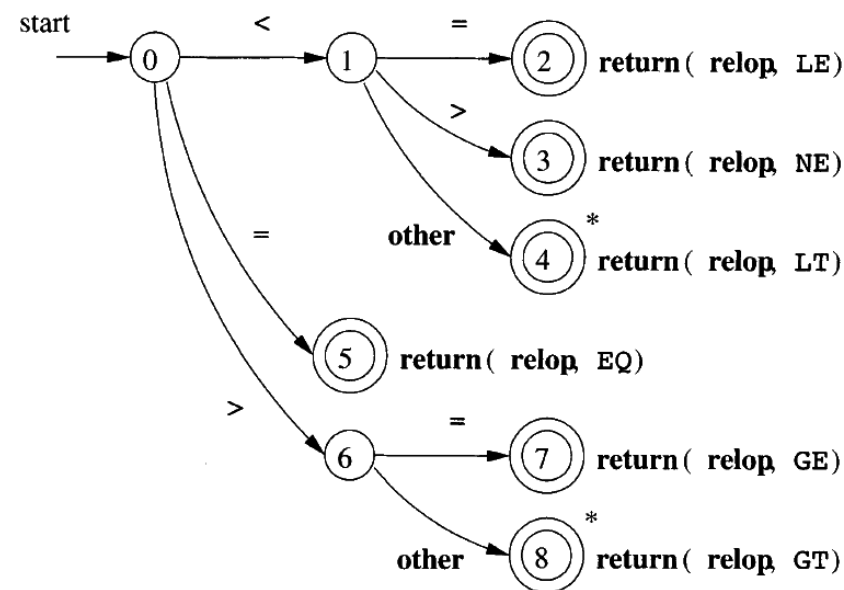
确定有穷自动机的实现

- 描述：正则表达式
- 两种方法
 - 基于词法单元的**词法结构图**或其它描述，手工编写代码扫描输入中的每个词素，并返回识别到的词法单元信息
 - 使用词法分析器生成工具（如lex /flex/ antlr），给出描述词素的**模式**，利用工具编译为具有词法分析器功能的代码，高效且简单

确定有穷自动机的实现

- 从转换图构造词法分析器的方法

- 变量state记录当前状态
- 一个switch根据state的值转到相应的代码
- 每个状态对应于一段代码
 - 这段代码根据读入的符号，确定下一个状态
 - 如果找不到相应的边，则调用fail()进行错误恢复
- 进入某个接受状态时，返回相应的**词法单元**
 - 注意状态有*标记时，需要回退forward指针

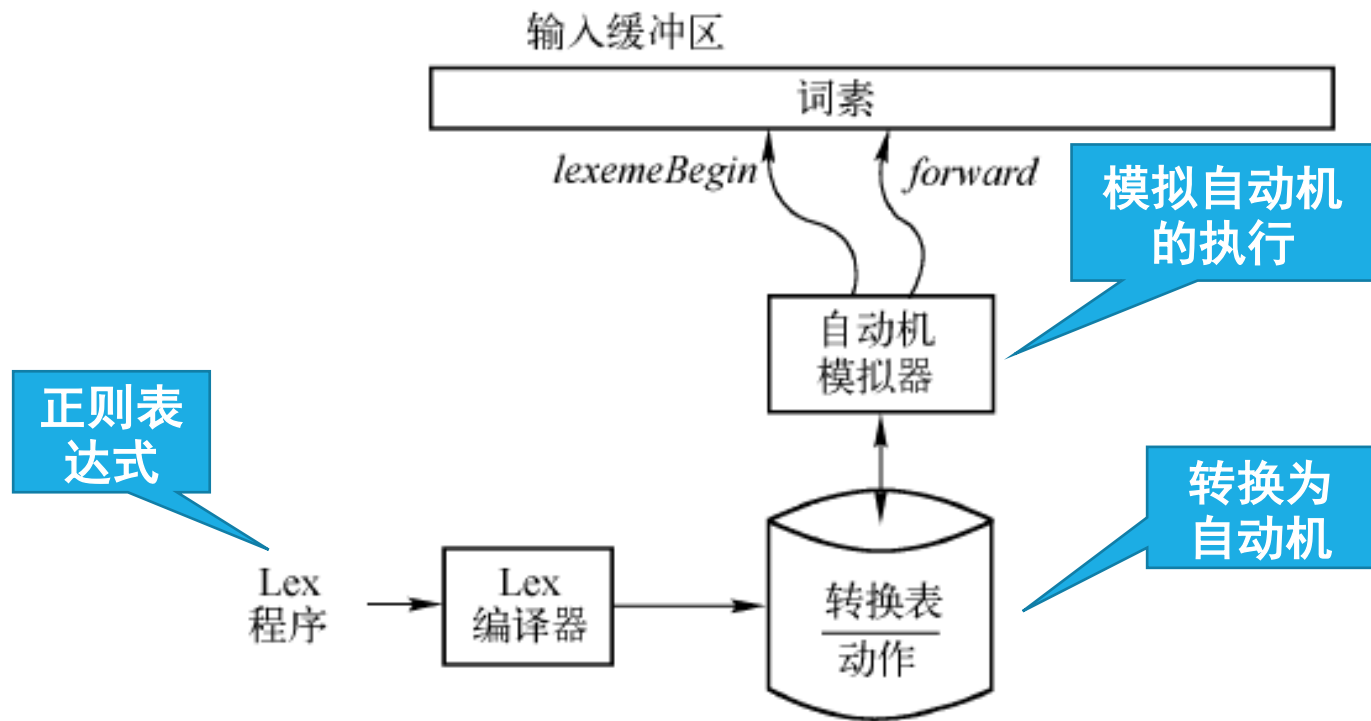


确定有穷自动机的实现

- 多个模式集成到词法分析器
- 方法1：词法分析器需要匹配多个模式（有多个状态转换图）
 - 顺序的尝试各个词法单元的状态转换图，如果引发fail，回退并启动下一个状态转换图
 - 次序问题
 - 优先级
- 方法2：并行的运行各个状态转换图
 - 一个图已经匹配到词素，另一个仍在继续读入
 - 取最长的和某个模式匹配的输入前缀（词法单元）
- 方法3：所有的状态转换图合并为一个图
 - 选择策略同方法2
 - 书中例子简单，因为没有两类词法单元以相同的字符开头

确定有穷自动机的实现

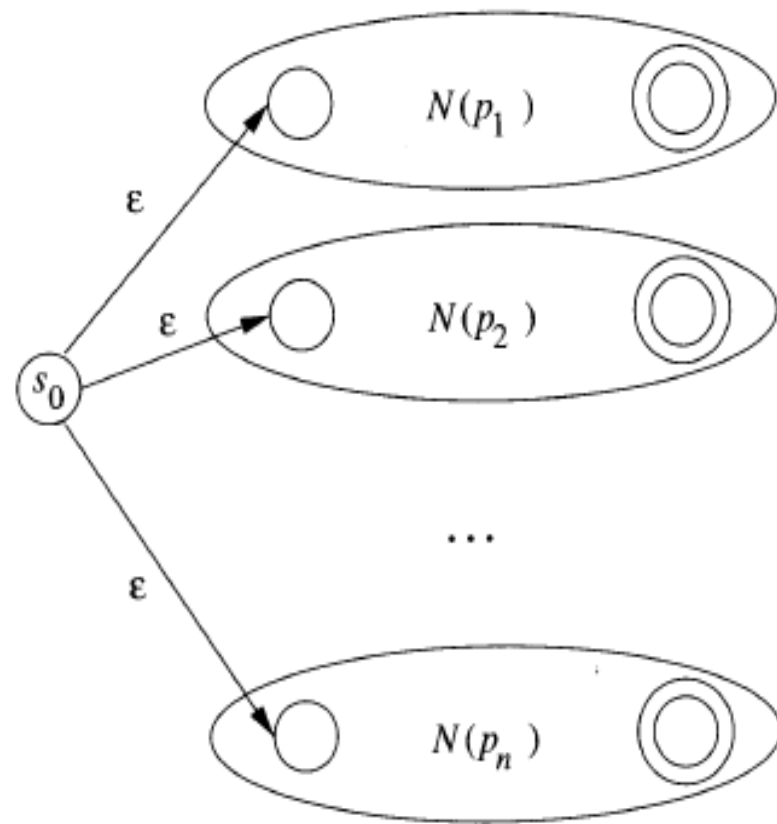
- 词法分析器生成工具的体系结构



确定有穷自动机的实现

• 自动机的构建

- 把Lex程序中的每个正则表达式转换成NFA
- 由于自动机需要识别所有与Lex程序中的模式相匹配的词素，因此我们需要将这些NFA合并为一个NFA
- 引入一个新的开始状态，从这个新状态到各个对应于模式 p_i 的NFA N_i 的开始状态各有一个 ϵ 转换



确定有穷自动机的实现

- 自动机构建的冲突问题

a	{ 模式 p_1 的动作 A_1 }
abb	{ 模式 p_2 的动作 A_2 }
$a * b^+$	{ 模式 p_3 的动作 A_3 }

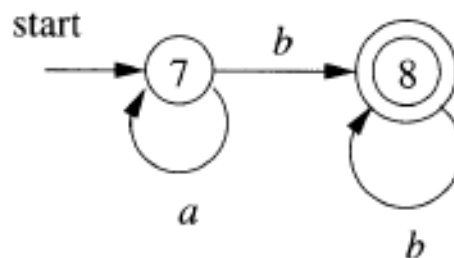
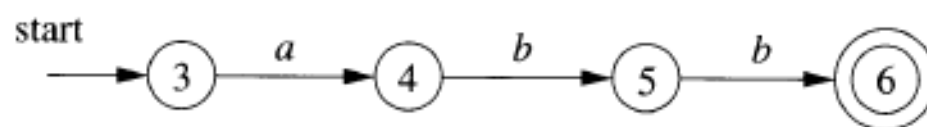
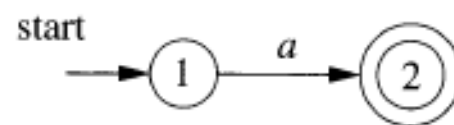
确定有穷自动机的实现

- 自动机构建的冲突问题

a { 模式 p_1 的动作 A_1 }

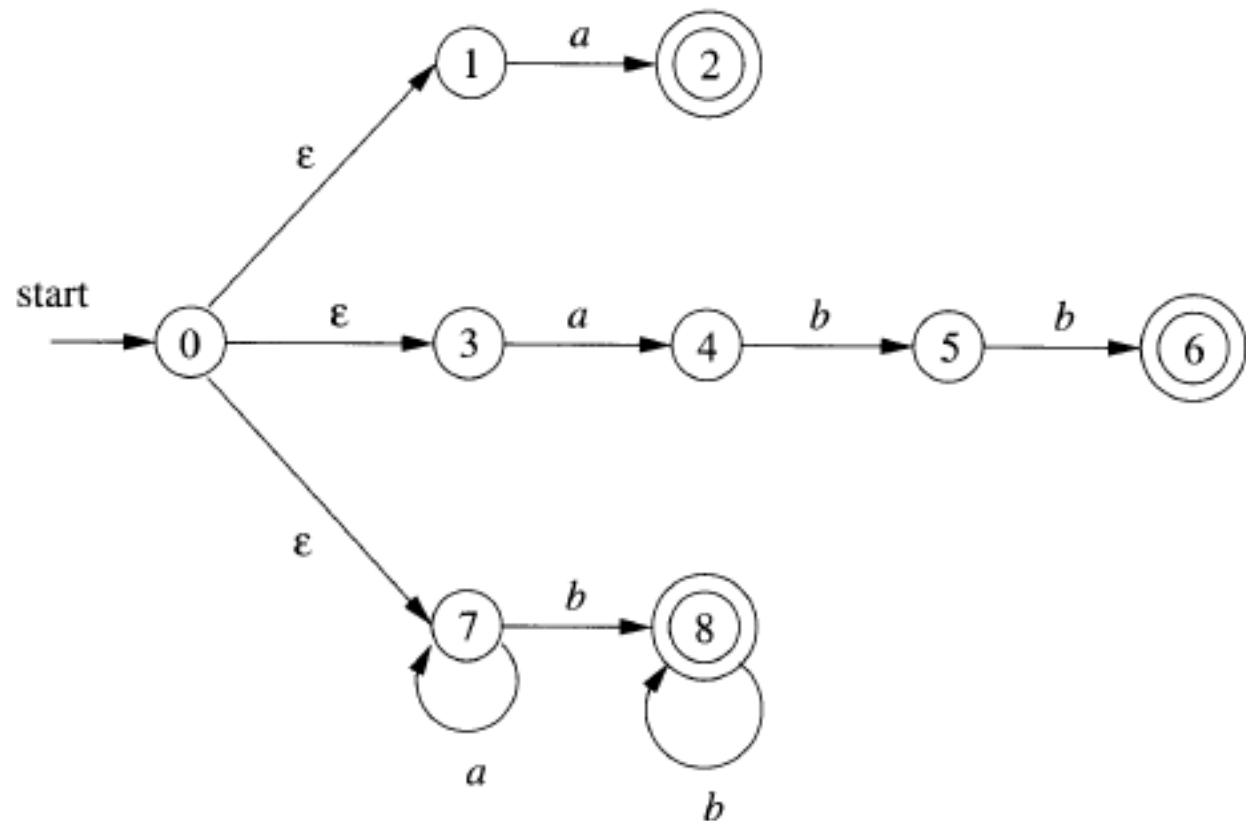
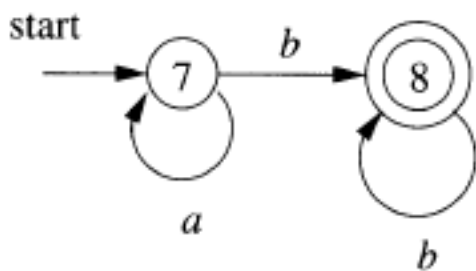
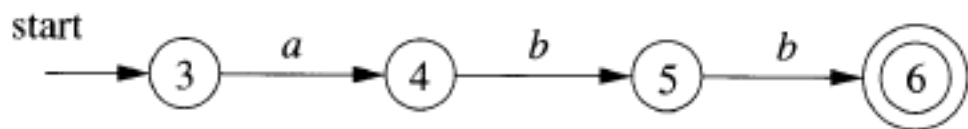
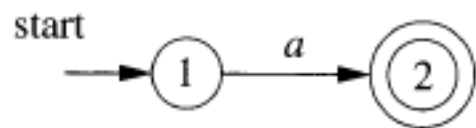
abb { 模式 p_2 的动作 A_2 }

a * b⁺ { 模式 p_3 的动作 A_3 }



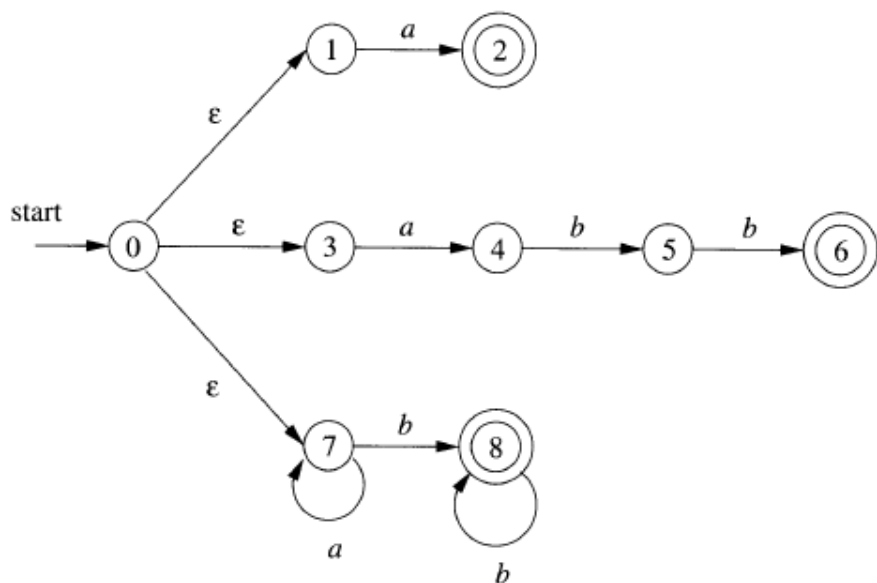
确定有穷自动机的实现

- 自动机构建的冲突问题



确定有穷自动机的实现

- 词法分析器模拟NFA的运行，直到到达一个没有后续状态的输入点
- 沿着状态集顺序回找，直到找到一个包含一个或多个接受状态的集合为止。如果集合中有多个接受状态，我们就选择和Lex程序中位置**最靠前的模式相关联的接受状态** p_i ，执行相应对应 A_i



如果要识别串：abba

a	{ 模式 p_1 的动作 A_1 }
abb	{ 模式 p_2 的动作 A_2 }
$a * b^+$	{ 模式 p_3 的动作 A_3 }

1. 词法分析

冯洋

南京大学计算机学院

fengyang@nju.edu.cn